

Simple Medical Appointments

Second Milestone - Software Testing - INSO 4117

Prof. Marko Schütz-Schmuck

Table of Contents

1. Informative part	2
1.1. Team	2
1.2. Current Situation, Needs, Ideas	4
1.3. Scope, Span, and Synopsis	9
1.4. Other Activities than Just Developing Source Code	10
1.5. Derived Goals	11
2. Descriptive Part	13
2.1. Domain Description	13
2.2. Requirements	26
2.3. Implementation	45
3. Analytic Part	51
3.1. Concept Analysis	51
3.2. Validation and Verification	54
3.3. Testing Methodology	58
4. Specific Class Topics	74
4.1. TDD	74
4.2. BDD and BDD tool usage	78
4.3. Use of debugging	82
Appendix	86
AI Disclosure Statement	86
Logbook	88

Notation for Changes

Notation	Purpose	Color	Example
[.added]# <text> #	Added	Green	This text is new.
[.changed]# <text> #	Paraphrased	Orange	This text has been modified.
[.removed]# <text> #	Removed	Red	This text is no longer relevant.

Chapter 1. Informative part

1.1. Team

The project for Milestone 1 is organized into 5 functional teams: Roles and Permissions, Clinical Experience, Patient Experience, Scheduling, and Documentation; plus 2 Managers. Each team is responsible for a specific dimension of the domain problem: simplifying medical appointment coordination in Puerto Rico. The structure reflects the need to address role and permission rules across patients, doctors, and clinic staff, clinical workflow representation, patient interaction, appointment logistics, and documentation coherence in a coordinated and structured manner.

The teams operate under a shared objective: to reduce friction in medical appointment scheduling and to improve accessibility, clarity, and administrative efficiency for both patients and healthcare providers. In this context, all team members are considered partners. No external partners have been formally identified for this milestone. However, potential future partners may include clinic administrators or healthcare professionals. If such partners arise, their roles and responsibilities will be documented accordingly. The lists are presented in alphabetical order and indicate each group's Team Leader.

Managers:

The Managers are responsible for coordination, milestone alignment, and ensuring coherence across all functional teams. They supervise task quality, verify alignment with milestone goals, and ensure compliance with documentation standards.

Student	GitHub username
Ivan O. Morales	@ivxnmorxles
Segmar Rosado	@Yulio7

Roles and Permissions Team

The Roles and Permissions Team is responsible for defining the domain roles involved in the appointment process and the permissions attached to each one. Their responsibility is to describe how patients, doctors, and clinic staff participate in the domain, what information each role can manage, and which actions each role is allowed to perform.

This team focuses on role boundaries, access responsibilities, and permission rules that keep patient care, provider activities, and clinic operations clearly coordinated within the domain.

Student	GitHub username
Diego Perez Gandarillas (Team Leader)	@diegoperez16
Jerac S. Durán Hernández	@jeracduran1
Carlos Pepin	@carlospepin23
Viviana V. Ramos	@vivianaramos6

Student	GitHub username
Jonathan Rodriguez	@Jonathan-Rod
Pierre Vales	@pierreg-vales

Clinical Experience Team:

The Clinical Experience Team is responsible for representing the healthcare provider perspective within the domain. Their responsibility is to define how doctors, specialties, clinics, and consultation availability are conceptually structured.

This team ensures that clinical availability, specialization categories, and appointment constraints reflect real-world healthcare dynamics in Puerto Rico.

Student	GitHub username
Iralys Sánchez Milián (Team Leader)	@iralys-sanchez18
Omar A. Cordero Maldonado	@omar-cordero
Michael Martinez Curbelo	@Cabinetman
Alejandro E. López Rodríguez	@alejandro-lopez24
Jorge Peralta García	@PeraltaJorge9128
Jean P. I. Sánchez	@JeanSanchezFelix

Patient Experience Team:

The Patient Experience Team is responsible for reasoning about how patients search, select, and request medical appointments. Their responsibility is to define how needs such as specialty filtering, location selection, and appointment visibility are conceptualized within the domain.

This team focuses on clarity, accessibility, and usability from the patient's perspective.

Student	GitHub username
Sebastian G. Suarez Robles (Team Leader)	@SebastianSuarez0429
Axel Burgos Amengual	@aburgos2005
Pedro A. Morales	@pedromorales
Juan Ortiz Abouomar	@juan.ortiz92
Gerardo José Ruiz Barreto	@Gerardoruiz1
Ariana P. Rodriguez Mendez	@arianrodz21
Sebastian I Saliva Giron	@SebastianSaliva

Scheduling Team:

The Scheduling Team is responsible for the temporal and logistical dimension of the domain. Their responsibility is to define how appointment slots are represented, reserved, modified, or canceled.

This team ensures that scheduling constraints, availability conflicts, and real-time updates are conceptualized independently of implementation technology.

Student	GitHub username
Tasnim Said Khaber (Team Lead)	@tasnim2204
Natanael Batista Álvarez	@NathBat09
Marymar Maldonado Roche	@marymarmaldonado
Victoria Miranda Osuna	@vmmo
Keishlyany M Sanabria	@keisanabria
Julian Vivas	@julivivas

Documentation Team:

The Documentation Team ensures that all project documentation is well-structured, accurate, and consistently maintained. They are responsible for organizing the informative, descriptive, and analytic parts of the documentation so that domain concepts, requirements, and design decisions remain clear and traceable.

Student	GitHub username
Jean P. I. Sánchez (Team Lead)	@JeanSanchezFelix
Alejandro Lopez	@alejandro-lopez24
Keishlyany M Sanabria	@keisanabria
Julian Vivas	@julivivas

1.2. Current Situation, Needs, Ideas

1.2.1. Current Situation

Healthcare Access Pressure in Puerto Rico

Access to medical services in Puerto Rico has become increasingly strained in recent years due to structural pressures on the healthcare system. One major factor is the sustained migration of healthcare professionals away from the island, which has reduced the number of practicing physicians available to meet patient demand. The WHO recommends a ratio of 2.3 health workers per 1,000 population. According to the PR College of Physicians and Surgeons (PRCPS), there are only 9,000 practicing doctors in Puerto Rico. Per the PRCPS, since 2014 365-500 physicians have left the island [1]. Additionally, due to geographic isolation, limited infrastructure, and chronic underfunding of healthcare services, many clinics operate at or near full capacity and patients often experience delays when attempting to obtain appointments [2].

Scheduling Practices and Operational Constraints

Medical appointment scheduling in many clinics still relies heavily on manual coordination methods, particularly phone-based booking systems. These methods require administrative staff to

manage calendars, answer calls, and coordinate changes throughout the day. Problems arise when staff are overwhelmed, resulting in long hold times for patients and administrative burnout. When paired with missed calls and manual data entry practices, mistakes such as double-bookings and incorrect patient details being entered into the system are likely to occur [3]. These inefficiencies can leave appointment slots unused while other patients remain waiting for care.

Patient Experience and Information Fragmentation

From the patient perspective, the process of securing a medical appointment is often time-consuming and uncertain. Patients frequently must contact multiple clinics individually to find available times or appropriate specialists. For example, Medicaid beneficiaries reported seeing an average of 7 providers and 4 practices each year [4]. There is typically no unified way to view appointment availability across providers, making it difficult to compare options or plan medical visits efficiently. For Medicaid beneficiaries alone, the typical primary care physician has to coordinate with 229 other physicians in 117 practices [4]. Delays in securing consultations may result in postponed treatment and increased patient frustration.

Partial Digital Measures and Their Limitations

Some clinics have introduced limited digital measures, such as sending appointment reminders via text message or email. While these efforts can reduce missed appointments, they do not provide patients with interactive scheduling capabilities or visibility into broader availability. A review conducted by members of Sheffield Hallam University and the University of Sheffield found sufficient evidence to suggest that the performance of these types of reminders are suboptimal [5]. Because these solutions operate independently within individual clinics, they do not address the wider coordination challenges across the healthcare system.

Observed Pain Points

These systemic issues manifest in various ways for patients and providers alike, (from rough sketches; see 2.1.2 and 2.1.4 for analysis)

- Call saturation and multi-month wait times; patients resort to paper reminders (Carmen, 5 Feb 2026).
- Office-hour constraints force repeated walk-ins and extra trips (Miguel, multiple visits in Jan 2026).
- Caregiver juggling cascades when one appointment shifts (Elena, March 2026).
- Waitlist status is opaque, leaving patients uncertain (Natalia).
- Insurance acceptance is hard to verify; multiple calls before finding a match (Rosa, 14 Feb 2026).
- Staff rely on notebooks plus spreadsheets and memory to reassign cancellations (Laura).
- Clinics cannot see each other's calendars, so patients must phone hop (Jose).
- Unpredictable consult lengths and no-shows distort schedules; defensive double booking adds churn (Dr. Rivera, Dr. Morales).
- Authorization checks can stall bookings and cause released slots (Marcos).

References:

- [1] Health Policy OPEN. "Puerto Rican physician's recommendations to mitigate medical migration from Puerto Rico to the mainland United States", 2024. <https://www.sciencedirect.com/science/article/pii/S2590229624000091>
- [2] Arbona Health Hub. "When Seconds Count: Puerto Rico's Ongoing Emergency Healthcare Crisis", 2025. <https://arbonahealthhub.com/2025/08/04/when-seconds-count-puerto-ricos-ongoing-emergency-healthcare-crisis/>
- [3] Autonoly. "Telegram Patient Appointment Scheduling Automation Guide". <https://www.autonoly.com/integrations/automation/telegram/patient-appointment-scheduling>
- [4] National Library of Medicine. "Patients' and Providers' Views on Causes and Consequences of Healthcare Fragmentation in the Ambulatory Setting: a Qualitative Study" <http://pmc.ncbi.nlm.nih.gov/articles/PMC6544669/>
- [5] National Library of Medicine. "Appointment reminder systems are effective but not optimal: results of a systematic review and evidence synthesis employing realist principles" <https://pmc.ncbi.nlm.nih.gov/articles/PMC4831598/>

1.2.2. Needs

Patients

Patients are currently able to contact clinics individually, typically by phone, in order to request appointments or inquire about availability. However, this process requires repeated attempts across multiple providers, provides limited visibility into available options, and depends heavily on timing and response availability from clinic staff. As a result, patients are unable to efficiently determine which providers can meet their needs within a reasonable timeframe. Patients therefore need a more reliable way to identify and compare viable appointment options based on factors such as specialty, location, and availability without relying on repeated manual inquiries.

Clinic Administrative Staff

Clinic administrative staff are currently responsible for managing appointment scheduling through manual coordination methods, including handling phone calls, maintaining calendars, and processing changes throughout the day. While these methods allow schedules to be maintained, they require continuous attention and are prone to interruptions, errors, and inconsistencies when demand is high. This makes it difficult to maintain accurate schedules while simultaneously responding to patient requests. Administrative staff therefore need a way to manage scheduling activities that reduces dependence on constant manual intervention and minimizes the likelihood of coordination errors.

Providers

Providers are currently able to define their availability and conduct consultations according to clinic schedules. However, variability in appointment durations, patient attendance, and administrative coordination introduces uncertainty into daily operations and limits the effective use of available time. Additionally, providers often have limited visibility into how their availability is allocated or adjusted throughout the scheduling process. Providers therefore need a way to

ensure that their available time is utilized in a predictable and controlled manner while maintaining flexibility to adapt to changing conditions.

Development Stakeholders

Development stakeholders are currently able to implement isolated solutions for specific aspects of appointment scheduling, such as reminders or record management. However, these solutions typically operate independently and do not capture the full set of relationships and constraints present in the scheduling domain. This fragmentation makes it difficult to represent scheduling processes consistently or to evolve the system in a structured way. Development stakeholders therefore need a way to model and manage scheduling as a coherent domain, where entities, constraints, and interactions can be consistently represented and extended over time.

1.2.3. Ideas

Real-Time Agenda

- Live availability display
- Synchronization of schedule updates between patients and clinics
- Management of time blocks and appointment durations

Waitlist Feature

If a doctor's schedule is full, patients can join a waitlist and automatically receive a slot when cancellations occur.

Conflict Detection and Capacity Handling

Prevent double bookings and manage maximum appointment limits per time slot.

Appointment type Configuration

Allow clinics to assign different durations for specific procedures and restrict certain procedures to designated time blocks.

Staff Review and Confirmation

Appointments may enter a pending state for staff approval before being finalized, ensuring correct booking types and improved reliability.

Patient Appointment Limits

Limit the number of active appointments per specialist (e.g., one at a time) to prevent unnecessary reservation of multiple slots.

Clinic Constraints Configuration

- Allow institutes to define rules such as:
- No appointments on specific days

- No appointments after certain hours
- Custom availability restrictions

Accessibility Features

- Clear interface design
- Larger text options
- Simplified navigation
- Accessibility-friendly layout for elderly users

Advanced Filtering Options

- Filter by specialty
- Filter by location *Filter by accepted health insurance plans
- Filter by doctor availability
- Filter by services offered

Clear Calendar Separation

Display schedules per doctor and per location without overlapping all calendars simultaneously.

Data Privacy and Protection Measures

- Secure document storage
- Controlled access permissions
- Patient history visibility limited to authorized roles
- Change history tracking

HIPAA Compliance Considerations

Availability data must be handled in a way that does not expose confidential patient information.

Secure Authentication and Role Management

Distinct roles (patient, doctor, location, admin) with clearly defined permissions.

Automated Notifications

- Appointment confirmations
- Cancellation alerts
- Rescheduling updates
- Reminder messages

1.3. Scope, Span, and Synopsis

1.3.1. Scope & Span

Scope

The scope of this project lies within the broader domain of healthcare service coordination, specifically focusing on the organization and management of outpatient medical appointments. This includes the interaction between patients, providers, and administrative entities in the process of requesting, scheduling, modifying, and attending medical consultations.

Within this domain, scheduling is influenced by constraints such as provider availability, appointment capacity, referral requirements, and patient demand. These interactions occur within a larger healthcare ecosystem characterized by fragmented communication channels, varying levels of digital adoption, and resource limitations. The project is situated in the context of Puerto Rico, where healthcare access is affected by workforce availability, geographic distribution of services, and administrative coordination practices.

The scope therefore encompasses the general problem space of how medical appointments are coordinated, managed, and accessed within a constrained healthcare environment.

Span

The span of this project is limited to the design and prototyping of a system that models appointment scheduling within outpatient, private clinic settings. The focus is on representing the core interactions, constraints, and entities involved in scheduling consultations between patients and providers.

The project is restricted to an academic prototype and does not extend to full-scale system deployment or real-world integration. It does not include hospital systems, emergency care coordination, or large-scale healthcare infrastructure. Additionally, the system will not interact with external services such as insurance providers, electronic health record systems, or third-party scheduling platforms. No real patient data or protected health information will be used. All data represented within the system will be simulated for the purpose of demonstrating scheduling behavior and system interactions.

The span is therefore confined to a controlled, simplified representation of outpatient appointment scheduling, focusing on core domain behavior rather than complete system implementation.

1.3.2. Synopsis

This project addresses the difficulty of coordinating medical appointments within a constrained healthcare environment in Puerto Rico, where access to care is limited by provider availability, fragmented scheduling practices, and administrative overhead. The objective is to model and prototype a structured approach to appointment coordination that reflects the constraints and interactions observed in the domain.

To achieve this, the project requires multiple types of work spanning analysis, design, implementation, and validation. Domain engineering activities are used to identify and formalize

key concepts such as patients, providers, appointments, and scheduling constraints, ensuring that the system reflects real-world practices. Requirements engineering translates these domain observations into precise, traceable requirements that define system behavior.

System design and architecture planning establish how scheduling logic, interactions, and data structures are organized, followed by implementation activities that develop prototype-level frontend and backend components. Supporting this, database design is used to represent domain entities and their relationships in a consistent and extensible manner.

Verification and validation are performed through multiple approaches. These include Test-Driven Development and Behavior-Driven Development to guide implementation, as well as load testing, mutation testing, and formal verification techniques such as Hoare logic to evaluate correctness and robustness. Additional validation activities include consistency checks, traceability verification, and walkthrough-based analysis of domain scenarios.

Throughout the project, continuous documentation, version control, and team coordination activities support development by maintaining clarity, alignment, and traceability across all components. These activities are carried out in parallel and iteratively, ensuring that the system evolves in a structured and verifiable manner toward a coherent representation of the scheduling domain.

1.4. Other Activities than Just Developing Source Code

1.4.1. Domain Engineering

Domain engineering activities focus on identifying, analyzing, and formalizing the core concepts, constraints, and interactions present in medical appointment scheduling. This includes examining stakeholder perspectives, operational practices, and domain tensions in order to construct a consistent domain model. These activities are carried out through domain descriptions, concept identification, and interaction modeling, ensuring that the system-to-be represents real-world behavior rather than assumptions.

1.4.2. Requirements Engineering

Requirements engineering is necessary to translate domain observations into precise and verifiable system requirements. This involves identifying domain properties, defining constraints, and documenting requirements in a structured manner that supports traceability. The process includes organizing requirements into coherent categories, refining them iteratively, and ensuring alignment with the domain model so that implementation decisions are grounded in clearly defined behavior.

1.4.3. System Design and Architecture

System design and architecture activities define how domain concepts and requirements are structured within the system. This includes the organization of components, the definition of interfaces, and the representation of scheduling logic and data relationships. Architectural decisions are guided by the need to maintain modularity, scalability, and consistency with the domain, enabling the system to evolve without violating its underlying structure.

1.4.4. Verification and Validation

Verification and validation activities ensure that the system behaves according to its specified requirements and accurately reflects the domain. This includes the use of Test-Driven Development and Behavior-Driven Development to guide implementation, as well as load testing, mutation testing, and formal verification techniques such as Hoare logic to evaluate correctness. Additional validation is performed through consistency checks, traceability verification, and scenario-based walkthroughs derived from domain interactions.

1.4.5. Documentation and Knowledge Management

Documentation activities support the development process by maintaining a clear record of domain descriptions, requirements, design decisions, and validation results. This includes structured documentation, version control practices, and continuous updates to ensure that all project artifacts remain consistent and accessible. These activities enable coordination among team members and provide a basis for evaluating design and implementation choices.

1.4.6. User Experience and Accessibility Considerations

User experience and accessibility considerations address the need to ensure that interactions with the system are understandable and usable across different stakeholder groups. These activities involve analyzing how patients, administrative staff, and providers interact with scheduling processes, and ensuring that representations of these interactions remain clear and consistent. Accessibility considerations are incorporated to account for variations in user capabilities and levels of digital literacy within the domain context.

1.5. Derived Goals

The primary goal of this project is to improve access to medical appointments in Puerto Rico by addressing scheduling inefficiencies and difficulty, below we will explore additional goals that arise from the nature of the problem.

Derived Goal 1: Reduce Administrative Friction in Healthcare Coordination

Clinics rely on manual coordination, phone based scheduling, and informal priority handling. These practices increase staff workload and reduce operational clarity. Beyond building a scheduling platform, a derived goal is to systematically reduce coordination friction between patients, providers, and administrative staff. This goal focuses not only on patient convenience but on redistributing cognitive and logistical load away from manual processes toward structured coordination mechanisms.

Derived Goal 2: Increase transparency of Availability Without Exposing Sensitive Data

The domain analysis identified limited visibility into appointment availability as a core issue. Patients repeatedly contact clinics without knowing whether appointments exist. At the same time, healthcare systems must protect confidential information. A derived goal is to increase real-time visibility of scheduling capacity while preserving privacy and role-based access boundaries. This aligns usability with data protection, ensuring transparency does not compromise confidentiality.

Derived Goal 3: Improve Fairness and Equity in Appointment Allocation

The domain revealed structural inequalities:

- Transportation limitations
- Regional disparities
- Informal priority lists
- Patients calling multiple clinics simultaneously

A derived goal is to promote equitable access to appointment opportunities through structured and transparent allocation mechanisms. This includes minimizing favoritism, informal redistribution, and arbitrary priority handling.

Derived Goal 4: Improve Patient Confidence and Trust in Scheduling Outcomes

The domain observations revealed uncertainty:

- Patients call repeatedly.
- Appointments are months away.
- Cancellations are informally reassigned.
- Elderly patients depend on relatives.

Beyond improving speed, a derived goal is to increase predictability and trust in appointment scheduling outcomes.

Trust emerges when: - Appointment states are clear. - Changes are communicated. - Priority rules are consistent. - Information is reliable.

Derived Goal 5: Support Long-Term Maintainability and Evolution

The current healthcare pressures in Puerto Rico are not temporary. Physician migration, demographic shifts, and digital adaptation will continue to evolve.

Thus, a derived goal is:

To establish a conceptual and architectural foundation that allows future expansion without redesigning core structures.

This includes the ability to later incorporate: - Analytics - Multi-clinic coordination - Advanced prioritization - Integration with external systems

By combining the primary goal of centralized appointment coordination with these derived goals, the project moves beyond building a scheduling tool and toward establishing: A structured, transparent, and equitable coordination framework for managing scarce healthcare resources. This forward-looking vision emphasizes sustainability, fairness, reliability, and collaborative system evolution ensuring that the project contributes not only to immediate scheduling improvements but to long-term modernization of medical appointment coordination practices.

Chapter 2. Descriptive Part

2.1. Domain Description

2.1.1. Domain Rough Sketches

NOTE

This section contains unprocessed observations collected from domain interviews with patients, clinic administrative staff, healthcare providers, and an insurance coordinator. No interpretive analysis appears here — that belongs in section 3.1 (Concept Analysis). The observations are recorded as close to verbatim as possible.

Patients

Patient (Carmen, 67, retired teacher):

"I called the clinic at 8:00 in the morning like they told me, but the line was already busy.

I kept dialing for almost an hour. When someone finally answered, they said the next available appointment was in three months.

I wrote the date on a small paper and stuck it on my fridge so I wouldn't forget."

Patient (Miguel, 34, construction worker):

"I can't always call during work hours. By the time I get out, the office is closed.

So I just go in person sometimes and ask if there's anything available.

If they say no, I just try again another day."

Patient (Elena, 52, caring for elderly mother):

"I manage my mother's appointments because she doesn't use a smartphone.

Sometimes I have to coordinate her primary doctor, a specialist, and lab tests.

If one appointment changes, everything else has to move too."

Patient (Natalia, university student):

"I joined a waitlist once, but nobody ever told me if I moved up or not.

I didn't know whether to wait or book somewhere else."

Patient (Rosa, 29, new resident in the area):

"I didn't know which doctor accepted my insurance.

I called three clinics before finding one that did.

There's no single place where you can compare everything."

Clinic Administrative Staff

Clinic Administrative Assistant (Laura, small family clinic):

"The phones don't stop ringing from the moment we open. Sometimes we're booking appointments while answering another call at the same time.

If someone cancels, we try to call another patient from memory, but not everyone picks up.

We use a notebook and a spreadsheet, just to be safe."

Clinic Administrator (José, regional clinic network):

"We don't have visibility into other clinics' schedules.

Patients call asking if another location has earlier availability, but we can't see that information.

Everything is isolated."

Doctors / Providers

Doctor (Dr. Rivera, cardiologist):

"My schedule looks full weeks ahead, but sometimes there are no-shows.

Other days, appointments take longer than expected, and everything runs late.

Patients get frustrated, but we can't predict exactly how long each consultation will take."

Doctor (Dr. Morales, general practitioner):

"Some patients book multiple appointments just in case, then cancel last minute.

I understand why they do it — they're afraid of losing the slot — but it makes scheduling unpredictable."

Insurance / External Coordination

Insurance Coordinator (Marcos, hospital billing office):

"Sometimes we have to confirm insurance authorization before finalizing an appointment.

If approval takes too long, the slot might already be given to someone else.

Patients think it's our fault, but we're waiting on external systems."

2.1.2. Terminology

NOTE

Each term below is derived from raw observations in the Domain Rough Sketch (2.1.1) and refined through concept analysis. Terms are presented with their

classification (entity, actor, event, behavior, function, etc.) and the phase in which they appear (domain, requirements, implementation).

Terms are organized into Domain Nouns, Domain Verbs, and Domain Constraints to align with ubiquitous language principles.

Domain Nouns (entities, actors, abstractions)

Patient

(actor, domain) A person seeking medical consultation or treatment. A patient may request, confirm, cancel, or reschedule appointments. **Derived from:** Carmen's redial marathon to secure a May slot (5 Feb 2026) and Natalia's stalled waitlist (Feb 2026).

Caregiver

(actor, domain) A person managing appointments on behalf of a patient who cannot coordinate independently. **Derived from:** Elena coordinating three linked visits for her mother in March 2026.

Healthcare Provider

(actor, domain) A licensed professional offering medical consultations (e.g., general practitioner, specialist). **Derived from:** Dr. Rivera's variable consult durations and no-shows (Feb 2026 sketch).

Doctor

(actor, domain) A specific type of healthcare provider with a medical degree. Doctors may have different scheduling constraints than other providers. **Derived from:** provider-level constraints voiced by Dr. Rivera and Dr. Morales (Feb 2026 sketches).

Clinic

(entity, domain) A healthcare facility that hosts providers and manages appointment availability within defined operational constraints.

Appointment

(entity, domain) A reserved time block allocated to a patient for consultation with a provider at a specific clinic location.

Time Slot

(entity, domain) A discrete, reservable interval in a provider's schedule. A time slot may be available, reserved, canceled, or expired.

Schedule

(entity, domain) The structured allocation of time slots associated with a provider or clinic.

Consultation Type

(taxonomy, domain) A classification of medical visit (e.g., follow-up, first-time visit, procedure) that influences duration and constraints.

Waitlist

(entity, domain) An ordered collection of patients waiting for a time slot when no immediate availability exists. **Derived from:** Natalia's opaque position after joining a waitlist (Feb 2026).

Cancellation

(event, domain) The release of a previously reserved appointment before its scheduled time.

No-Show

(event, domain) The absence of a patient at a scheduled appointment without prior cancellation.

Derived from: Dr. Rivera's no-show fluctuation (Feb 2026 sketch).

Insurance Authorization

(process, domain) An external approval required before certain appointments can be confirmed.

Derived from: Marcos waiting on insurer portals before confirming slots (Feb 2026 sketch).

Referral

(entity/event, domain) A directive from one provider requiring a patient to consult another provider before treatment.

Availability

(state, domain) The condition of a time slot being open for reservation.

Booking Status

(state, domain) The current condition of an appointment (pending, confirmed, canceled, completed).

Operational Hours

(constraint entity, domain) The defined time boundaries within which a clinic accepts or schedules appointments. **Derived from:** Miguel finding the clinic closed after his shift (21-22 Jan 2026).

Domain Verbs (actions, events, domain operations)

Request Appointment

(action, domain) A patient expresses intent to reserve an available time slot. **Derived from:** Carmen repeatedly calling at 8:00 a.m. to request a slot (5 Feb 2026).

Reserve Time Slot

(action, domain) The act of assigning a specific time slot to a patient.

Confirm Appointment

(event, domain) The transition of an appointment from pending to confirmed state.

Cancel Appointment

(action/event, domain) The intentional release of a reserved time slot by patient or clinic.

Reschedule Appointment

(action, domain) The replacement of one reserved time slot with another.

Join Waitlist

(action, domain) A patient registers interest in being assigned a slot if one becomes available.

Derived from: Natalia joining a waitlist without receiving position updates (Feb 2026).

Assign Waitlisted Patient

(event, domain) The allocation of a newly freed time slot to the next eligible patient in the waitlist.

Authorize Coverage

(external event, domain) Insurance approval required before final confirmation of certain appointment types. **Derived from: Marcos pausing confirmations until insurer approval arrives (Feb 2026).**

Expire Slot

(event, domain) Automatic release of a temporarily held slot after a defined confirmation window.

Domain Constraints (invariant business rules)

No Double Booking

A single time slot cannot be reserved by more than one patient simultaneously. **Derived from: Dr. Morales observing duplicate bookings that cancel late (Feb 2026 sketch).**

Single Active Appointment per Provider Constraint

A patient may not hold multiple concurrent active appointments with the same provider.

Operational Hour Restriction

Appointments cannot be scheduled outside clinic operational hours. **Derived from: Miguel's inability to book after 6:00 p.m. when the clinic door was locked (22 Jan 2026).**

Consultation Duration Integrity

The allocated time slot must match the required duration for the selected consultation type.

Waitlist Ordering Integrity

Waitlisted patients must be assigned based on defined priority rules.

Authorization Dependency

Certain appointments cannot transition to confirmed status without required insurance approval. **Derived from: Marcos losing slots while waiting for insurer authorization (Feb 2026 sketch).**

Cross-Phase Terms (Explicitly Annotated)

These terms appear in later phases and are included for clarity.

User Account

(entity, implementation) A technical authentication construct representing a system login identity. **Not a domain concept.**

Authentication Token

(artifact, implementation) A signed credential used to verify session identity.

Notification

(system event, requirements) A system-generated alert informing patients or clinics about

appointment changes.

Session

(technical state, implementation) The authenticated runtime context allowing authorized actions.

Core Domain Flow (Appointment Lifecycle)

1. A provider defines availability through structured time slots within operational hours.
2. A patient requests an appointment by selecting an available time slot.
3. If constraints apply (authorization or referral), the appointment enters pending status.
4. Upon confirmation, the slot becomes reserved.
5. If canceled, the slot returns to available state.
6. If a waitlist exists, the next patient may be assigned automatically.
7. The appointment either completes successfully or becomes a no-show.
8. Historical records preserve appointment outcomes for traceability.

2.1.3. Narrative

The medical appointment scheduling domain involves the coordination of patients, caregivers, healthcare providers, doctors, clinics, schedules, time slots, and appointments to organize consultation services. A healthcare provider, such as a doctor, offers consultations within a clinic and defines availability through a schedule composed of discrete time slots constrained by operational hours. Each time slot has an availability state and may transition through reserved, canceled, or expired conditions as part of its lifecycle.

The process begins when a patient, or a caregiver acting on their behalf, requests an appointment for a specific consultation type. Clinic staff review the schedules of available providers and reserve an appropriate time slot, creating an appointment that links the patient, provider, clinic, and scheduled time. The appointment is assigned a booking status, such as pending or confirmed, depending on whether constraints like insurance authorization or referral requirements apply. A patient may hold zero or more appointments but cannot hold multiple active appointments with the same provider simultaneously.

As appointments are confirmed, time slots transition from available to reserved, reducing overall availability within the schedule. A patient or clinic may cancel an appointment, triggering a cancellation event that returns the associated time slot to an available state. Patients may also reschedule appointments by releasing one reserved time slot and reserving another. In cases where no immediate availability exists, a patient may join a waitlist, which maintains an ordered collection of patients awaiting assignment. When a time slot becomes available, an assign waitlisted patient event may allocate the slot to the next eligible patient according to defined ordering constraints.

Clinic staff manage schedules manually, coordinating multiple providers across different consultation types while ensuring domain constraints such as no double booking, consultation duration integrity, and operational hour restrictions are maintained. If a provider becomes

unavailable, scheduled appointments must be rescheduled to alternative time slots or providers. Additionally, appointments may result in no-show events when patients fail to attend without prior cancellation, leaving reserved time slots unused. These interactions define how booking status, availability, and scheduling constraints evolve over time within the domain.

2.1.4. Events, Actions, and Behaviors

Appointment slot published

- **Type:** Event
- **Definition:** A clinic or provider makes an appointment slot available to the community.
- **Trigger:** A provider defines availability in terms of time, location, and service.
- **Observable Condition / Resulting State:** A new appointment slot exists with status **available** and is visible to patients for discovery and booking.
- Reference to sketch: Providers define working hours; Jose cannot see other sites' calendars to compare newly published slots.

Appointment requested

- **Type:** Action → Event
- **Definition:** A patient expresses intent to book a specific appointment slot.
- **Trigger:** A patient selects a slot and submits a request.
- **Observable Condition / Resulting State:** A request record exists; the appointment enters a confirmation or waitlist process depending on constraints.
- Reference to sketch: Carmen redialed 23 times starting 5 Feb 2026 at 8:02 a.m. to request a slot.

Appointment confirmed

- **Type:** Event
- **Definition:** An appointment request is validated and officially scheduled.
- **Trigger:** Clinic approval, automatic validation, or completion of confirmation workflow.
- **Observable Condition / Resulting State:** The slot status becomes **reserved**; the appointment is part of the clinic schedule and reminders may be scheduled.
- Reference to sketch: Clinics maintain structured schedules once slots are assigned.

Appointment canceled

- **Type:** Action → Event
- **Definition:** A previously confirmed or requested appointment is withdrawn.
- **Trigger:** A patient or clinic cancels the appointment.
- **Observable Condition / Resulting State:** The slot becomes **available** again; downstream processes (e.g., waitlist handling) may be triggered.
- Reference to sketch: Laura flips through a notebook and spreadsheet to reassign cancellations.

Slot reassigned to waiting patient

- **Type:** Event
- **Definition:** An available slot is allocated to a patient from a waiting list.
- **Trigger:** A cancellation or new availability combined with existing waitlist demand.
- **Observable Condition / Resulting State:** The slot becomes **reserved** for the selected patient; notification is issued.
- Reference to sketch: Informal priority lists used when earlier openings appear (same workflow Laura described).

Provider availability modified

- **Type:** Action → Event
- **Definition:** A provider updates their working schedule or availability constraints.
- **Trigger:** A provider changes working hours, location, or service capacity.
- **Observable Condition / Resulting State:** Availability records are updated; conflicting appointments may require rescheduling or notification.
- Reference to sketch: Providers split time across clinics and schedules change frequently; Miguel found doors locked after 6:00 p.m.

Appointment reminder issued

- **Type:** Action → Event
- **Definition:** A reminder notification is generated for an upcoming appointment.
- **Trigger:** The scheduled appointment time approaches based on configured timing rules.
- **Observable Condition / Resulting State:** A notification record exists; the patient has been informed through available communication channels.
- Reference to sketch: Staff sometimes send manual reminders from personal phones.

Patient marked absent

- **Type:** Event
- **Definition:** A patient fails to attend a scheduled appointment.
- **Trigger:** The appointment time passes without attendance confirmation.
- **Observable Condition / Resulting State:** The appointment is marked as **no-show**; attendance metrics are updated.
- Reference to sketch: Dr. Rivera reported fluctuating no-shows.

Referral recorded for consultation

- **Type:** Event (Entity creation)
- **Definition:** A referral authorizing a patient to access a consultation is registered.
- **Trigger:** An external provider issues and records a referral.

- **Observable Condition / Resulting State:** A referral entity exists; eligibility constraints for booking are satisfied.
- Reference to sketch: Referral requirements introduce coordination steps.

Patient joins waitlist

- **Type:** Action → Event
- **Definition:** A patient registers interest in future appointment availability when no slots are currently available.
- **Trigger:** A patient opts into a waitlist due to lack of availability.
- **Observable Condition / Resulting State:** A waitlist entry exists; the patient becomes eligible for reassignment when slots open.
- Reference to sketch: Natalia joined a waitlist in February 2026 and never received status updates.

Conceptual Function Signatures

Purpose: These conceptual functions describe the core domain operations from a system-independent perspective, including their purpose, required inputs, outputs, and dependencies. They map to the domain events and actions defined above.

Publish Slot

```
publishSlot(timeSlotCatalog: TimeSlotCatalog, provider: Provider, slot: TimeSlot):  
Result TimeSlotId Error
```

- **Description:** Adds a new appointment slot to the provider's availability and makes it discoverable.
- **Conceptual Inputs:** Time slot catalog, provider, time slot definition.
- **Conceptual Output:** Identifier of the created slot or an error.
- **Dependencies:** Requires provider authorization and valid scheduling constraints.
- **Related Domain Events/Actions:** Corresponds to **Appointment slot published** (event).

Request Appointment

```
requestAppointment(appointmentBook: AppointmentBook, patient: Patient, slotId:  
TimeSlotId): Result AppointmentId Error
```

- **Description:** Registers a patient's request to book a specific appointment slot.
- **Conceptual Inputs:** Appointment book, patient, slot identifier.
- **Conceptual Output:** Appointment identifier or error.
- **Dependencies:** Slot must exist and be available.

- **Related Domain Events/Actions:** Corresponds to **Appointment requested** (action/event).

Confirm Appointment

```
confirmAppointment(appointmentBook: AppointmentBook, timeSlotCatalog: TimeSlotCatalog,  
referralRegistry: ReferralRegistry, insuranceRegistry: InsuranceRegistry,  
appointmentId: AppointmentId): Result Appointment Error
```

- **Description:** Validates and confirms an appointment based on constraints such as referrals and insurance.
- **Conceptual Inputs:** Appointment book, slot catalog, referral and insurance registries, appointment ID.
- **Conceptual Output:** Confirmed appointment or error.
- **Dependencies:** Requires eligibility validation and slot availability.
- **Related Domain Events/Actions:** Corresponds to **Appointment confirmed** (event).

Cancel Appointment

```
cancelAppointment(appointmentBook: AppointmentBook, timeSlotCatalog: TimeSlotCatalog,  
appointmentId: AppointmentId): Result Unit Error
```

- **Description:** Cancels an existing appointment and releases its associated slot.
- **Conceptual Inputs:** Appointment book, slot catalog, appointment ID.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Appointment must exist and be cancelable.
- **Related Domain Events/Actions:** Corresponds to **Appointment canceled** (action/event).

Reschedule Appointment

```
rescheduleAppointment(appointmentBook: AppointmentBook, timeSlotCatalog:  
TimeSlotCatalog, appointmentId: AppointmentId, newSlotId: TimeSlotId): Result Unit  
Error
```

- **Description:** Changes an existing appointment to a different available slot.
- **Conceptual Inputs:** Appointment book, slot catalog, appointment ID, new slot ID.
- **Conceptual Output:** Success or failure.
- **Dependencies:** New slot must be available and compatible.
- **Related Domain Events/Actions:** Impacts **Appointment confirmed** and **Provider availability modified**.

Modify Availability

```
modifyAvailability(schedule: Schedule, provider: Provider, range: DateTimeRange, policy: Policy): Result Unit Error
```

- **Description:** Updates a provider's working hours or scheduling constraints.
- **Conceptual Inputs:** Schedule, provider, time range, policy.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Must respect scheduling policies and avoid conflicts.
- **Related Domain Events/Actions:** Corresponds to **Provider availability modified** (action/event).

Join Waitlist

```
joinWaitlist(waitlist: Waitlist, patient: Patient, specialty: Specialty): Result Unit Error
```

- **Description:** Registers a patient in a waitlist for a given specialty.
- **Conceptual Inputs:** Waitlist, patient, specialty.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Requires absence of immediate availability.
- **Related Domain Events/Actions:** Corresponds to **Patient joins waitlist** (action/event).

Select Waitlist Candidate

```
selectWaitlistCandidate(waitlist: Waitlist, slot: TimeSlot): Option Patient
```

- **Description:** Selects an eligible patient from the waitlist for a newly available slot.
- **Conceptual Inputs:** Waitlist, time slot.
- **Conceptual Output:** Optional patient.
- **Dependencies:** Depends on prioritization rules.
- **Related Domain Events/Actions:** Supports **Slot reassigned to waiting patient** (event).

Reassign Slot

```
reassignSlot(waitlist: Waitlist, appointmentBook: AppointmentBook, timeSlotCatalog: TimeSlotCatalog, notificationQueue: NotificationQueue, slotId: TimeSlotId): Option AppointmentId
```

- **Description:** Assigns an available slot to a waitlisted patient and notifies them.

- **Conceptual Inputs:** Waitlist, appointment book, slot catalog, notification queue, slot ID.
- **Conceptual Output:** Optional appointment ID.
- **Dependencies:** Requires candidate selection and slot availability.
- **Related Domain Events/Actions:** Corresponds to **Slot reassigned to waiting patient** (event).

Queue Reminder

```
queueReminder(notificationQueue: NotificationQueue, appointment: Appointment):  
NotificationQueue
```

- **Description:** Adds a reminder notification for a scheduled appointment.
- **Conceptual Inputs:** Notification queue, appointment.
- **Conceptual Output:** Updated notification queue.
- **Dependencies:** Appointment must be confirmed.
- **Related Domain Events/Actions:** Supports **Appointment reminder issued** (action/event).

Issue Reminder

```
issueReminder(notificationQueue: NotificationQueue, channel: Channel): Result Unit  
Error
```

- **Description:** Sends queued reminders through a communication channel.
- **Conceptual Inputs:** Notification queue, channel.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Requires configured communication channels.
- **Related Domain Events/Actions:** Corresponds to **Appointment reminder issued** (event).

Mark No-Show

```
markNoShow(appointmentBook: AppointmentBook, appointmentId: AppointmentId): Result  
Unit Error
```

- **Description:** Marks an appointment as unattended.
- **Conceptual Inputs:** Appointment book, appointment ID.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Triggered after appointment time passes.
- **Related Domain Events/Actions:** Corresponds to **Patient marked absent** (event).

Record Referral

```
recordReferral(referralRegistry: ReferralRegistry, patient: Patient, provider:
Provider, referral: Referral): Result Unit Error
```

- **Description:** Registers a referral required for consultation eligibility.
- **Conceptual Inputs:** Referral registry, patient, provider, referral.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Requires external provider input.
- **Related Domain Events/Actions:** Corresponds to **Referral recorded for consultation** (event).

Record Insurance Approval

```
recordInsuranceApproval(insuranceRegistry: InsuranceRegistry, patient: Patient,
provider: Provider, appointmentId: AppointmentId): Result Unit Error
```

- **Description:** Stores insurance approval for a scheduled appointment.
- **Conceptual Inputs:** Insurance registry, patient, provider, appointment ID.
- **Conceptual Output:** Success or failure.
- **Dependencies:** Requires insurer validation.
- **Related Domain Events/Actions:** Supports **Appointment confirmed** (event).

Search Providers

```
searchProviders(system: ClinicSystem, specialty: Specialty): List Provider
```

- **Description:** Retrieves providers matching a given specialty.
- **Conceptual Inputs:** Clinic system, specialty.
- **Conceptual Output:** List of providers.
- **Dependencies:** None (public operation).
- **Related Domain Events/Actions:** Supports discovery behavior.

List Available Slots

```
listAvailableSlots(timeSlotCatalog: TimeSlotCatalog, provider: Provider): List
TimeSlot
```

- **Description:** Lists available slots for a provider.
- **Conceptual Inputs:** Slot catalog, provider.
- **Conceptual Output:** List of time slots.
- **Dependencies:** Slots must be published.

- **Related Domain Events/Actions:** Supports **Appointment slot published**.

Authorize Action

```
authorizeAction(policy: Policy, actor: Actor, action: Action): Result Unit Error
```

- **Description:** Determines whether an actor is allowed to perform an action.
- **Conceptual Inputs:** Policy, actor, action.
- **Conceptual Output:** Authorization result.
- **Dependencies:** Depends on role and policy definitions.
- **Related Domain Events/Actions:** Governs all domain actions.

Booking Behavior

```
bookingBehavior(system: ClinicSystem, patient: Patient): BehaviorTrace
```

- **Description:** Represents the full interaction flow of a patient booking an appointment.
- **Conceptual Inputs:** Clinic system, patient.
- **Conceptual Output:** Behavior trace.
- **Dependencies:** Composes multiple domain functions.
- **Related Domain Events/Actions:** Maps to booking-related behavior scenarios.

2.2. Requirements

2.2.1. User Stories, Epics, Features

NOTE

This subsection defines the product scope of the Simple Medical Appointments system from a user-value perspective. It organizes the solution into Epics that capture high-level healthcare goals and Features that realize those goals in the system.

Abbreviations and ID Conventions:

Abbrev.	Meaning
US	User Story: a user-centered need framed as intent and value.
E	Epic: a high-level goal that groups related features and stories.
F	Feature: a concrete capability that realizes part of an epic.
ReqRef	Requirement Reference: the requirement ID(s) a story or feature maps to.
REQ	Requirement: a functional or non-functional specification with a stable ID.

Identifier Formats:

Type	Format	Components	Example
User Story ID:	US.AREA.NN	AREA = functional area (AUTH, PROF, AVAIL, BOOK, SCHED, NOTIF, ADMIN)	US.BOOK.01
Epic ID:	E# or “Epic E#: Title”	# = epic number	E1
Feature ID:	F#.N	# = epic number; N = feature sequence	F1.1
Requirement ID:	REQ-AREA-TYPE-NN	TYPE = F (Functional) or NF (Non-functional)	REQ-BOOK-F-01

Functional Requirement Definitions (ReqRef)

Req ID	Description	Linked Epics/Features
REQ-AUTH-F-01	Authenticate users, create accounts, and enforce role-based access (patient, doctor, admin). Provide clear failure reasons and lockout on repeated failures.	E1 → F1.1, F1.4
REQ-PROF-F-01	Persist patient and doctor profile data (name, contact, specialty/clinic, accessibility prefs). Validate required fields, allow users to update their own profiles, and audit changes.	E1 → F1.2, F1.3
REQ-AVAIL-F-01	Allow providers to declare weekly availability and ad-hoc slots; prevent overlapping slots and surface conflicts to the user.	E2 → F2.1, F2.2, F2.3
REQ-BOOK-F-01	Let patients book available slots, reserve the slot atomically, and prevent double booking.	E3 → F3.2, F3.3, F3.4
REQ-BOOK-F-02	Allow patients to cancel upcoming appointments and immediately release the slot for reuse.	E3 → F3.3
REQ-NOTIF-F-01	Send confirmation notifications after successful booking, including appointment details.	E4 → F4.1
REQ-NOTIF-F-02	Send reminder notifications before the appointment based on user preferences.	E4 → F4.2, F4.3
REQ-ADMIN-F-01	Provide admins with read-only oversight dashboards (appointments, users, activity) with filtering/export; enforce least-privilege access.	E5 → F5.1, F5.2

Epic E1: Trusted Care Identity & Profiles

Goal: Ensure patients and doctors are recognized securely so their appointments, history, and permissions travel with them.

Problem/value: Users need trusted identities and accurate profiles to receive the right care, see

their history, and communicate with providers.

Features (F1):

- F1.1 Account registration and login
- F1.2 Patient profile management
- F1.3 Doctor profile management
- F1.4 Role-based access control

US.AUTH.01: Be recognized securely | ReqRef: [REQ-AUTH-F-01]

"As a patient or doctor, I want the system to recognize me securely so that my appointments, notes, and permissions are tied to me across visits."

Acceptance criteria:

Given	When	Then
Valid registration data	The form is submitted	An account is created with the selected role.
Missing required fields	Submission is attempted	Inline validation errors are displayed.

US.PROF.01: Maintain my profile | ReqRef: [REQ-PROF-F-01]

"As a patient, I want to manage my personal information so that doctors have accurate details during appointments."

Acceptance criteria:

Given	When	Then
Profile page is open	Information is updated and saved	Changes are stored and timestamp updated.
Invalid data entered	Save is attempted	Validation errors prevent saving.

E1 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-AUTH-01	F1.1, F1.4	[REQ-AUTH-F-01]	Authentication and role logic.
US-PROF-01	F1.2, F1.3	[REQ-PROF-F-01]	Profile persistence and validation.

Epic E2: Doctor Availability & Scheduling

Goal: Allow doctors to define when they are available.

Problem/value: Appointment booking requires clearly defined time slots to prevent overlaps and

confusion.

Features (F2):

- F2.1 Define weekly availability
- F2.2 Create specific appointment slots
- F2.3 Prevent overlapping availability

US.AVAIL.01: Set my availability | ReqRef: [\[REQ-AVAIL-F-01\]](#)

"As a doctor, I want to define my available time slots so that patients can book appointments."

Acceptance criteria:

Given	When	Then
Availability form	Valid time range entered	Slot is saved and visible in calendar.
Overlapping time range	Save is attempted	System rejects with conflict message.

E2 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-AVAIL-01	F2.1, F2.2, F2.3	[REQ-AVAIL-F-01]	Calendar conflict rules.

Epic E3: Appointment Booking

Goal: Enable patients to book medical appointments easily.

Problem/value: Patients need a simple way to find available doctors and reserve time slots without manual coordination.

Features (F3):

- F3.1 Search doctors by specialty
- F3.2 View available time slots
- F3.3 Book appointment
- F3.4 Prevent double booking

US.BOOK.01: Book an appointment | ReqRef: [\[REQ-BOOK-F-01\]](#)

"As a patient, I want to book an available time slot so that I can secure a consultation with a doctor."

Acceptance criteria:

Given	When	Then
An available slot exists	Booking is confirmed	Appointment is created and slot becomes unavailable.
Slot already booked	Booking attempted	System prevents double booking.

US.BOOK.02: Cancel an appointment | ReqRef: [\[REQ-BOOK-F-02\]](#)

"As a patient, I want to cancel my appointment so that the time slot becomes available again."

Acceptance criteria:

Given	When	Then
Existing future appointment	Cancel is confirmed	Appointment is removed and slot reopens.
Past appointment	Cancel attempted	System prevents cancellation.

E3 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-BOOK-01	F3.2, F3.3, F3.4	[REQ-BOOK-F-01]	Booking transaction and conflict handling.
US-BOOK-02	F3.3	[REQ-BOOK-F-02]	Cancellation logic.

Epic E4: Notifications & Reminders

Goal: Reduce missed appointments.

Problem/value: Patients and doctors benefit from timely reminders to avoid no-shows.

Features (F4): - F4.1 Appointment confirmation notification - F4.2 Automated reminders before appointment - F4.3 Notification preferences

US.NOTIF.01: Receive appointment confirmation | ReqRef: [\[REQ-NOTIF-F-01\]](#)

"As a patient, I want to receive confirmation after booking so that I know my appointment is secured."

Acceptance criteria:

Given	When	Then
Appointment created	Booking succeeds	Confirmation notification is sent.

US.NOTIF.02: Receive appointment reminder | ReqRef: [\[REQ-NOTIF-F-02\]](#)

"As a user, I want to receive a reminder before my appointment so that I do not forget it."

Acceptance criteria:

Given	When	Then
Appointment is 24 hours away	Reminder job runs	Reminder notification is delivered.

E4 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-NOTIF-01	F4.1	[REQ-NOTIF-F-01]	Confirmation trigger.
US-NOTIF-02	F4.2, F4.3	[REQ-NOTIF-F-02]	Reminder scheduler and preferences.

Epic E5: Administrative Oversight

Goal: Maintain system integrity and operational control.

Problem/value: Administrators must monitor users, appointments, and resolve conflicts.

Features (F5):

- F5.1 View all users
- F5.2 View all appointments
- F5.3 Deactivate accounts

US.ADMIN.01: Monitor appointments | ReqRef: [\[REQ-ADMIN-F-01\]](#)

"As an administrator, I want to view all appointments so that I can ensure system integrity."

Acceptance criteria:

Given	When	Then
Admin dashboard	Appointments page opened	All appointments are listed with filters.

E5 Traceability (Stories → Features → Requirements):

Story ID	Feature	ReqRef	Notes
US-ADMIN-01	F5.1, F5.2	[REQ-ADMIN-F-01]	Admin read access and filtering.

2.2.2. Personas

The personas below represent our core user segments and ground the scope of this product. For each persona we outline goals, pains, typical behaviors, and accessibility needs, and we link them to the stories, epics, and features defined in 2.2.1.

We will reference these personas by name during sprint planning and reviews (e.g., “Would this work for Carmen?”) to ensure decisions remain user-centered and concrete.

Table 1. Carmen “Busy Mom” Rodríguez – Narrative Persona

Snapshot	Carmen is the kind of person who manages three WhatsApp groups before 8 a.m., prepares breakfast while reviewing homework, and still remembers her mother's blood pressure medication schedule by heart.
Background	38-year-old elementary school teacher and mother of two children (ages 6 and 10). Lives in a mid-sized city and helps coordinate medical appointments not only for her children but also for her elderly mother who lives nearby.
Daily Life Context	Her day is tightly scheduled between teaching, commuting, helping with homework, and managing household responsibilities. Most personal tasks are deferred to late evenings when everything is quieter.
Wishes & Goals	Wants to book pediatric and general checkups quickly without making phone calls during school hours. She hopes for a simple system that lets her see available times, book in seconds, and receive reminders so she does not forget appointments.
Fears & Pains	Frustrated by long waiting times on the phone and unclear availability. She has previously shown up to a clinic only to discover the appointment was misrecorded. She fears double-booking appointments across family members.
Motivations	Driven by responsibility toward her family and a desire to stay organized despite constant interruptions.
Decision Tendencies	Chooses convenience and clarity. If a system looks confusing or requires too many steps, she postpones the task. She values clear confirmation messages and calendar synchronization.
Healthcare Habits	Books most appointments in the evening after putting her children to bed. Keeps a handwritten planner but relies increasingly on digital reminders.
Tech & Environment	Uses her smartphone as her primary device. Often books appointments late at night from the living room couch with moderate Wi-Fi reliability.
Visual Cue	Carmen at the kitchen table with a planner open, phone in hand, comparing school schedules with doctor availability.
Domain Role	Represents the Primary Patient / Family Coordinator who needs fast booking, clear confirmations, and reliable reminders.

Table 2. Dr. Rafael “Overbooked Specialist” Morales – Narrative Persona

Snapshot	Dr. Morales moves briskly between consultation rooms, dictating notes while adjusting his glasses, always aware that the waiting room is filling up.
Background	45-year-old cardiologist working in both a private clinic and a public hospital. Balances patient consultations, administrative paperwork, and occasional university lectures.

Daily Life Context	His schedule is fragmented across locations, often running behind due to emergencies or complex cases. Administrative tasks compete with patient care for his time.
Wishes & Goals	Wants a structured way to define his weekly availability and prevent overlapping appointments. Hopes to reduce no-shows and last-minute confusion.
Fears & Pains	Frustrated by double bookings caused by manual scheduling. Dislikes administrative interruptions during consultations. Worries about wasted time due to missed appointments.
Motivations	Seeks efficiency to focus more on patient care rather than administrative overhead.
Decision Tendencies	Values efficiency and predictability. Prefers systems that automate routine scheduling logic and reduce manual intervention.
Healthcare Habits	Plans his availability weekly, adjusting based on surgeries, conferences, or hospital duties.
Tech & Environment	Uses a desktop computer at the clinic and a tablet between appointments. Expects systems to be fast and intuitive due to limited time.
Visual Cue	Dr. Morales reviewing his digital calendar between patients, quickly approving available slots before the next consultation begins.
Domain Role	Embodies the Doctor who needs availability control, conflict prevention, and structured scheduling tools.

Table 3. Marta “Clinic Administrator” López – Narrative Persona

Snapshot	Marta sits at the clinic’s front desk with a headset on, answering calls while checking patient files and reminding the next person in line to fill out a form.
Background	52-year-old clinic administrator with over 20 years of experience. Responsible for managing appointment records, assisting patients, and coordinating between doctors.
Daily Life Context	Her workday is fast-paced and interruption-heavy, with constant switching between phone calls, in-person requests, and administrative updates.
Wishes & Goals	Wants a centralized dashboard where she can view all appointments, quickly identify conflicts, and assist patients who call for help.
Fears & Pains	Overwhelmed when multiple patients call at once to reschedule. Fears data inconsistencies and lost records.
Motivations	Pride in keeping the clinic organized and ensuring patients are attended to efficiently.
Decision Tendencies	Relies on structured lists and clear filters. Prefers systems that show everything at a glance and allow quick corrections.

Healthcare Habits	Confirms next-day appointments manually by phone when possible, especially for elderly patients.
Tech & Environment	Works on a desktop computer with multiple windows open; occasionally checks updates on her phone.
Visual Cue	Marta leaning toward her monitor, scanning a full-day schedule while highlighting names on a printed backup list.
Domain Role	Represents the Administrative Staff who needs oversight, filtering, and reliable appointment records.

Table 4. José “Elderly Patient” Hernández – Narrative Persona

Snapshot	José carries a small notebook in his jacket pocket with handwritten appointment dates and medication notes written in careful script.
Background	67-year-old retired mechanic living with his wife. Suffers from hypertension and requires regular follow-ups with his general practitioner.
Daily Life Context	His routines are stable and predictable, centered around home life, errands, and medical care. He values independence but occasionally relies on family support.
Wishes & Goals	Wants to easily see when his next appointment is and receive reminders without navigating complicated interfaces.
Fears & Pains	Feels uncomfortable with complex technology. Afraid of pressing the wrong button and losing information.
Motivations	Wants to remain independent and not rely entirely on his daughter for scheduling.
Decision Tendencies	Trusts systems that look simple and readable. Prefers large text and clear confirmation messages.
Healthcare Habits	Relies on his daughter to help him with online tasks but wants to maintain independence where possible.
Tech & Environment	Uses a basic Android phone with enlarged font settings.
Visual Cue	José sitting in a quiet armchair, carefully reading a reminder notification before marking the date in his notebook.
Domain Role	Embodies the Accessibility-Sensitive Patient who needs simplicity, clarity, and reassurance.

Table 5. Lucía “University Student” Vega – Narrative Persona

Snapshot	Lucía studies late into the night with headphones on, switching between lecture slides and messaging friends about upcoming exams.
Background	21-year-old university student living in a shared apartment. Frequently postpones medical appointments due to busy academic schedules.
Daily Life Context	Her schedule is irregular, driven by classes, assignments, and social activities. She often multitasks and prioritizes urgent academic deadlines.

Wishes & Goals	Wants to book appointments outside class hours and receive reminders synced with her digital calendar.
Fears & Pains	Often forgets appointments when overwhelmed by exams. Frustrated by limited appointment visibility.
Motivations	Values flexibility and autonomy in managing her time.
Decision Tendencies	Books quickly if the system shows real-time availability. Cancels or reschedules if conflicts arise.
Healthcare Habits	Prefers online systems over phone calls and avoids administrative interactions unless necessary.
Tech & Environment	Primarily uses her smartphone; expects fast loading and intuitive navigation.
Visual Cue	Lucía booking an appointment from her dorm bed while checking her exam schedule on another app.
Domain Role	Represents the Digitally-Native Patient who values speed, flexibility, and mobile-first design.

Table 6. Ricardo “System Administrator” Castillo – Narrative Persona

Snapshot	Ricardo reviews dashboards with a cup of strong coffee, watching system logs scroll across his screen.
Background	34-year-old IT administrator responsible for maintaining clinic software systems, data integrity, and backups.
Daily Life Context	Balances reactive problem-solving (incidents, bugs) with proactive maintenance and monitoring. Often works outside normal hours to avoid disruptions.
Wishes & Goals	Wants system stability, secure user authentication, and accurate appointment logs.
Fears & Pains	Concerned about data breaches, system downtime, and inconsistent records.
Motivations	Driven by reliability and professional responsibility to keep systems running smoothly.
Decision Tendencies	Prefers structured data, audit trails, and role-based access control.
Healthcare Habits	Monitors system activity during peak hours and performs maintenance at night.
Tech & Environment	Works on high-performance desktop systems with multiple monitoring tools open.
Visual Cue	Ricardo analyzing system metrics late in the evening to ensure smooth performance the next day.
Domain Role	Embodies the Technical Administrator ensuring security, reliability, and compliance.

2.2.3. Domain Requirements

Appointment Slot Requirements

Slot Capacity

Domain basis: Consultation periods are limited by time, staff, and facility availability.

Requirements:

- REQ-SLOT-F-01: The system must associate each appointment slot with a finite capacity defined by the clinic or provider.
- REQ-SLOT-F-02: The system must prevent allocation of appointments beyond the defined capacity of a slot.

Appointment Association

Domain basis: Consultations occur between a patient and a healthcare professional and form part of the patient's medical history.

Requirements:

- REQ-APPT-F-01: The system must associate each scheduled appointment with exactly one patient and at least one provider.
- REQ-APPT-F-02: The system must maintain the association between a patient and their scheduled appointments over time.

Provider Availability Requirements

Availability Representation

Domain basis: Providers work within defined schedules that vary by clinic and day.

Requirements:

- REQ-AVAIL-F-02: The system must ensure that appointments only occur within declared availability intervals.

Appointment Confirmation Requirements

Confirmation Rules

Domain basis: Some consultations depend on insurance authorization, referrals, or administrative review, so provisional bookings do not guarantee consultation.

Requirements:

- REQ-CONFIRM-F-01: The system must represent whether an appointment requires confirmation before becoming final.
- REQ-CONFIRM-F-02: The system must prevent appointments requiring confirmation from being treated as finalized until confirmation has occurred.

Appointment Reassignment Requirements

Cancellation and Reassignment

Domain basis: Canceled time periods become available for reassignment to reduce patient waiting time.

Requirements:

- REQ-REASSIGN-F-01: The system must provide means to reassign released appointment slots to other patients.

Attendance Tracking Requirements

Domain basis: Clinics monitor attendance to evaluate workload, reliability, and resource use; past participation influences future scheduling and clinical planning.

Requirements:

- REQ-ATTEND-F-01: The system must record whether a patient attended a scheduled appointment.
- REQ-ATTEND-F-02: The system must preserve attendance records as part of appointment history.

Referral and Authorization Requirements

Domain basis: Certain specialist visits cannot occur without prior approval or referral.

Requirements:

- REQ-REFERRAL-F-01: The system must represent when a consultation depends on an external referral or authorization.
- REQ-REFERRAL-F-02: The system must prevent scheduling of consultations requiring referral until such referral is recorded.

Appointment Sequencing Requirements

Domain basis: Consultations occur in sequence and cannot overlap for the same provider.

Requirements:

- REQ-SEQUENCE-F-01: The system must represent appointment time as an ordered temporal position within a schedule.
- REQ-SEQUENCE-F-02: The system must prevent overlapping appointments for a single provider.

2.2.4. Interface Requirements

Data Initialization Interfaces

Initialization Mechanisms

Domain basis: The system must provide mechanisms to initialize internal representations of domain entities before scheduling operations. These mechanisms support:

- Patients and Providers, who require reliable account creation and booking.
- Providers, who need structured availability control.
- Clinics, which depend on consistent records.
- System-level identity, which requires stable integrity management.

Requirements:

- IR-DI-01: The system must provide mechanisms to initialize internal representations of domain entities before scheduling operations.

Provider Creation

Domain basis: The system allows authorized users to create Provider records. A Provider must be associated with at least one Clinic. Provider identity attributes must be immutable after creation. Only authenticated users with appropriate roles ([US-AUTH-01], F1.4) may perform this action.

Requirements:

- IR-DI-02: The system allows authorized users to create Provider records.

Clinic Creation

Domain basis: The system allows creation of Clinic entities. Clinics should maintain a collection of associated Providers. Administrative access must be restricted according to role-based policies.

Requirements:

- IR-DI-03: The system allows creation of Clinic entities.

Time Slot Definition

Domain basis: The system allows Providers to define Time Slots ([US-AVAIL-01]). Each Time Slot has:

- Provider reference
- Clinic reference
- Start time
- End time
- Status (Available, Reserved, Expired)

A Time Slot must be initialized with status = Available. Overlapping time ranges should be rejected (F2.3, [REQ-AVAIL-F-01]).

Requirements:

- IR-DI-04: The system allows Providers to define Time Slots.

Patient Records

Domain basis: The system allows creation of Patient records ([US-AUTH-01]). Required fields must be validated prior to persistence. Patient contact information must be editable ([US-PROF-01]). Accessibility considerations must support simplified interaction needs.

Requirements:

- IR-DI-05: The system allows creation of Patient records.

Appointment Records

Domain basis: Appointment records include:

- Referral status
- Insurance authorization status

Default status must be Pending when applicable. This supports structured confirmation logic and external coordination workflows.

Requirements:

- IR-DI-06: Appointment records must include referral and insurance authorization status.

Appointment Scheduling Interfaces

Appointment Creation

Domain basis: The system creates an Appointment with status = Requested. A Time Slot must not be reserved until confirmation conditions are satisfied.

Requirements:

- IR-ES-01: Appointment creation sets status = Requested and preserves slot availability until confirmation.

Appointment Confirmation

Domain basis: The system verifies:

- Time Slot availability
- Referral requirements
- Insurance authorization (if required)

Upon validation, the system sets Appointment status = Confirmed and associated Time Slot status = Reserved. A confirmation notification must be triggered (F4.1).

Requirements:

- IR-ES-02: The system validates appointment conditions and sets status accordingly.

Appointment Cancellation

Domain basis: The system must set Appointment status = Cancelled. The associated Time Slot must immediately revert to Available. Slot release occurs atomically with status update.

Requirements:

- IR-ES-03: Cancelled appointments must release slots atomically.

Appointment Rescheduling

Domain basis: The system releases the previously reserved Time Slot, reserves the new Time Slot, and updates the Appointment time reference. These operations must be executed atomically to support flexible rescheduling and reduced coordination burden for Patients.

Requirements:

- IR-ES-04: The system must allow atomic reassignment of appointment slots.

Appointment No-Show

Domain basis: The system must set Appointment status = No-Show. The Time Slot remains Reserved for historical traceability. Audit logging supports system integrity requirements.

Requirements:

- IR-ES-05: Appointment status = No-Show must be recorded without releasing the Time Slot.

Insurance Status Updates

Domain basis: The system updates insuranceStatus upon receipt of approval. Confirmation must only proceed if the slot remains Available. This supports structured oversight for administrative processes.

Requirements:

- IR-ES-06: Insurance status updates must be respected before confirmation.

Time Slot and State Interfaces

Time Slot Integrity

Domain basis: A Time Slot may be associated with at most one Appointment. A Time Slot with status ≠ Available should not be reservable. Concurrent reservation attempts must be serialized. Double-booking must be prevented at the data layer (F3.4). Slot state transitions should be logged for traceability.

Requirements:

- IR-TI-01: Time Slot objects must enforce single appointment association and serialized

reservation.

State Transitions

Domain basis: Valid transitions include:

- Requested → Confirmed
- Confirmed → Completed
- Confirmed → Cancelled
- Confirmed → No-Show

Invalid transitions must be rejected with appropriate error responses. Completed and No-Show appointments must be immutable except for administrative annotations.

Requirements:

- IR-ST-01: Appointment state transitions must follow the defined rules.

Data Mutability

Domain basis: Mutable Data:

- Patient contact information ([\[US-PROF-01\]](#))
- Appointment notes
- Referral status prior to confirmation

Immutable Data:

- appointmentID
- Historical completed appointments
- Provider identity after confirmation

Unauthorized modification attempts return an error response, supporting security and integrity goals.

Requirements:

- IR-EC-01: Data mutability must respect rules for mutable and immutable attributes.

User Interface Interfaces

UI Behavior

Domain basis: Availability Visibility:

- The UI must display available Time Slots without exposing patient data.
- Slot availability must update dynamically after state changes.

Appointment Transparency:

- The UI clearly displays current Appointment status.
- Status changes must be reflected immediately after confirmation.

Notification Feedback:

- The UI must confirm successful booking, cancellation, or rescheduling ([US-NOTIF-01]).
- Error messages should be specific and validation-driven.

Role-Based Access:

- UI actions must be restricted according to authenticated role ([US-AUTH-01], F1.4).
- Administrative views support oversight needs.

Requirements:

- IR-UI-01: The system UI must reflect availability, appointment status, notifications, and enforce role-based access.

API Interfaces

POST /appointments

Domain basis: POST /appointments ([US-BOOK-01]) creates Appointment with status = Requested and validates required fields.

Requirements:

- IR-API-01: Endpoint must validate input and create appointments.

PUT /appointments/{id}/confirm

Domain basis: PUT /appointments/{id}/confirm validates slot availability and dependencies and sets Appointment status and Time Slot atomically.

Requirements:

- IR-API-02: Endpoint must confirm appointments atomically.

PUT /appointments/{id}/cancel

Domain basis: PUT /appointments/{id}/cancel ([US-BOOK-02]) sets Appointment status = Cancelled and releases associated Time Slot.

Requirements:

- IR-API-03: Endpoint must cancel appointments and release slots.

GET /timeslots

Domain basis: GET /timeslots?provider={id} ([US-AVAIL-01]) returns only Available Time Slots. All endpoints enforce authentication via JWT ([US-AUTH-01]), validate input schema, and return

structured error responses (400, 401, 422, 409).

Requirements:

- IR-API-04: Endpoint must return available slots and enforce authentication.

Architecture Requirements

Layered Request Flow

Domain basis: The architecture separates Authentication, Validation, Business Logic, and Repository layers. Request flow must be Authentication → Validation → Business Logic → Repository → Response. Handlers cannot bypass validation/authorization, and atomic scheduling logic must execute in the Business Logic layer.

Requirements:

- IR-CC-01: System architecture must enforce layered request flow and atomic scheduling logic.

2.2.5. Machine Requirements

Machine Initialization Requirements

Resource Preparation

Domain basis: System operations depend on properly initialized machine resources, including memory, threads, and data structures.

Requirements:

- REQ-MACH-F-01: The system must initialize all machine-level resources before execution of scheduling and booking operations.
- REQ-MACH-F-02: The system must create and validate Provider process threads, associating each with at least one Clinic process, with immutable identity attributes.
- REQ-MACH-F-03: The system must instantiate Clinic data structures, maintaining references to associated Provider threads, with access restricted by system roles.

Time Slot Object Requirements

Slot Initialization

Domain basis: Time slots represent available consultation periods and must prevent conflicts at the machine level.

Requirements:

- REQ-MACH-SLOT-F-01: The system must initialize Time Slot objects with Provider and Clinic references, start and end times, and a default status of Available.
- REQ-MACH-SLOT-F-02: The system must reject overlapping time ranges at the machine level.

Patient Object Requirements

Patient Initialization

Domain basis: Patient objects must be consistent and editable for core operations.

Requirements:

- REQ-MACH-PAT-F-01: The system must initialize Patient objects with required fields validated before memory allocation.
- REQ-MACH-PAT-F-02: Patient contact information must be mutable after object creation, and accessibility constraints enforced.

Appointment Object Requirements

Appointment Creation

Domain basis: Appointments must be represented accurately in machine objects to support scheduling workflows.

Requirements:

- REQ-MACH-APPT-F-01: Appointment objects must include referral and insurance authorization status, defaulting to Pending.
- REQ-MACH-APPT-F-02: Appointment creation must set status = Requested, and associated Time Slot objects remain unreserved until validation succeeds.

Appointment Confirmation

Domain basis: Some appointments depend on insurance authorization, referrals, or administrative review.

Requirements:

- REQ-MACH-CONFIRM-F-01: The system must verify slot availability, referral requirements, and insurance authorization before confirming an appointment.
- REQ-MACH-CONFIRM-F-02: Upon validation, the system must set Appointment status = Confirmed and associated Time Slot status = Reserved atomically.

Appointment Cancellation and Reassignment

Domain basis: Canceled appointments free resources for reassignment to maintain efficient scheduling.

Requirements:

- REQ-MACH-REASSIGN-F-01: The system must cancel Appointment objects by setting status = Cancelled and revert associated Time Slot objects to Available atomically.
- REQ-MACH-REASSIGN-F-02: The system must support atomic reassignment of previously reserved Time Slot objects to new appointments.

Time Slot and Appointment Constraints

State and Serialization Rules

Domain basis: System must prevent double-booking and maintain traceability of state transitions.

Requirements:

- REQ-MACH-TI-F-01: Each Time Slot object may be linked to at most one Appointment object.
- REQ-MACH-TI-F-02: Concurrent reservation attempts must be serialized and double-booking prevented at the data layer.
- REQ-MACH-ST-F-01: Valid state transitions include Requested → Confirmed → Completed, Confirmed → Cancelled, Confirmed → No-Show; invalid transitions must generate error responses.

Data Mutability and Security

Attribute Control

Domain basis: Certain attributes must remain immutable to maintain system integrity, while others remain editable for workflow operations.

Requirements:

- REQ-MACH-EC-F-01: Mutable attributes include Patient contact information, Appointment notes, and referral status prior to confirmation.
- REQ-MACH-EC-F-02: Immutable attributes include Appointment ID, historical completed appointments, and Provider ID after confirmation; unauthorized modification attempts must generate system errors.

2.3. Implementation

2.3.1. Selected Fragments of the Implementation

The following fragments illustrate how key architectural decisions are reflected in the implementation. Rather than presenting raw code for its own sake, each fragment is accompanied by an explanation of the reasoning behind it and how it supports the system's design goals.

Application Initialization and Structure

The backend is initialized using an application factory pattern, implemented in `backend/app/init.py`.

```
def create_app(testing: bool = False):
    app = Flask(__name__)

    app.config.from_object(Config)
    if testing:
        app.config["TESTING"] = True
```

```
CORS(app)
init_supabase(app)
register_routes(app)

return app
```

This pattern allows the application to be created in a controlled and flexible way. Instead of relying on global initialization, the system builds the application step by step. Configuration is loaded first, followed by external service initialization and route registration.

This separation is important because it allows different environments to be handled cleanly. For example, when running tests, the application can disable external integrations such as Supabase. It also makes the system easier to maintain, since each responsibility is clearly defined and isolated.

Route Organization

The API is structured using blueprints, defined in `backend/app/routes/init.py`.

```
def register_routes(app):
    app.register_blueprint(main_bp, url_prefix='/api')
    app.register_blueprint(appointment_bp, url_prefix='/api')
    app.register_blueprint(profile_bp, url_prefix='/api/profile')
    app.register_blueprint(settings_bp, url_prefix='/api/settings')
```

This approach reflects a modular design where each functional area of the system is handled independently. For example, profile-related operations are separated from appointment-related operations.

This modularity reduces coupling between components and makes it easier to extend the system. New features can be added as new blueprints without modifying existing ones, which reduces the risk of introducing bugs in unrelated parts of the application.

Supabase Integration

The system integrates with Supabase for authentication and data storage. This integration is centralized in `backend/app/supabase.py`.

```
def init_supabase(app):
    global supabase

    if app.config.get("TESTING"):
        supabase = None
        app.supabase = None
        return None
```

```

url = app.config.get('SUPABASE_URL')
key = app.config.get('SUPABASE_KEY')

if not url or not key:
    raise RuntimeError('Supabase environment variables are missing.')

supabase = create_client(url, key)
app.supabase = supabase
return supabase

```

By initializing the client once and attaching it to the application, the system avoids duplicating configuration logic across different components.

An important detail is the conditional behavior during testing. When the application runs in testing mode, the Supabase client is disabled. This prevents external dependencies from affecting test execution and ensures that tests remain fast and reliable.

Backend Communication with Supabase

To maintain consistency, all communication with Supabase is handled through a wrapper function defined in `backend/app/routes/profile.py`.

```

def _supabase_request(...):
    headers = {
        'apikey': service_key,
        'Authorization': f"Bearer {user_token or service_key}",
    }

```

This abstraction centralizes how requests are constructed and executed. Instead of writing HTTP logic in every endpoint, the system delegates this responsibility to a single function.

This design improves maintainability. If the way requests are handled needs to change, it can be updated in one place without affecting the rest of the codebase. It also ensures consistent error handling and response parsing across all interactions with Supabase.

Authentication and Authorization

Authentication and authorization are handled as separate steps, both implemented in `backend/app/routes/profile.py`.

```

status, user_response, _ = _supabase_request('GET', '/auth/v1/user', user_token=token)

role_status, role_response, _ = _supabase_request(
    'GET',
    '/rest/v1/user_roles',
    query={'user_id': f'eq.{user_id}', 'select': 'roles(name)'},
)

```

The system first verifies the identity of the user using a token. Once the user is authenticated, a second query retrieves the user's role.

This separation is intentional. Authentication answers the question “who is the user?”, while authorization answers “what is the user allowed to do?”. Keeping these concerns independent makes the system more flexible and easier to evolve as role definitions change.

Access Control in Profile Endpoints

Access control is enforced directly within the endpoints, which are defined in `backend/app/routes/profile.py`.

```
if not _can_access_target(request_user_id, request_role, user_id):
    return jsonify({'error': 'Forbidden'}), 403
```

This check ensures that users can only access or modify their own data unless additional permissions are granted.

Although simple, this explicit check is critical for system security. By placing it early in the request flow, the system prevents unauthorized access before any sensitive operations are performed.

Frontend API Communication

On the frontend, API communication is centralized in `frontend/src/services/api.js`.

```
const response = await fetch(`${BASE_URL}${path}`, {
  headers: { 'Content-Type': 'application/json', ...authHeaders },
  ...restOptions,
})
```

Before sending a request, the system retrieves the current session and attaches the authentication token automatically.

This design removes the need for individual components to handle authentication manually. As a result, the code becomes more consistent and less error-prone, since all requests follow the same pattern.

2.3.2. Role-Based UI Behavior

The frontend adapts its behavior based on the user's role, implemented in `frontend/src/pages/Home.jsx`.

```
const { data: roleRows } = await supabase
  .from('user_roles')
  .select('roles(name)')
  .eq('user_id', data.user.id)
```

Once the role is retrieved, it is used to control what the user sees in the interface. This creates a direct connection between backend authorization and frontend behavior. The system not only enforces permissions internally but also reflects them in the user interface, improving both usability and security.

Appointment Creation and Scheduling Logic

One of the core operations of the system is the creation of appointments. This process involves multiple steps, including authentication, validation, conflict detection, and persistence. The API endpoint responsible for this operation is implemented in `backend/app/routes/scheduling_appointments.py`.

```
@scheduling_appointments_bp.route("/", methods=["POST"])
@requires_auth
def create_appointment():
    data = _validate_json_request()
    supabase = _get_supabase_client()
    response = supabase.table("appointments").insert(data).execute()
    return jsonify({"success": True, "data": response.data}), 201
```

Before reaching the endpoint, the request is processed by authentication middleware defined in `backend/app/middleware/auth_middleware.py`.

```
def requires_auth(f):
    token = extract_token()
    payload = verify_jwt(token)
    g.user_id = payload.get("sub")
    return f()
```

This ensures that only authenticated users can create appointments, while keeping authentication logic separate from business logic.

Beyond the API layer, the system includes a dedicated module for handling scheduling rules, implemented in `backend/src/booking.py`.

```
if has_conflict(proposed_start, proposed_end, existing_appointments):
    return False, None, ["time_slot_conflict"]
```

This logic ensures that appointments do not overlap with existing ones. It represents the core business rule of the scheduling system. Additional constraints are enforced through availability and closure checks. Availability is retrieved from `backend/app/repositories/availability.py`, while closures are handled in `backend/app/services/closure_service.py`. These components ensure that appointments are only scheduled within valid time ranges and do not conflict with blocked periods such as holidays or doctor unavailability.

This flow demonstrates how different layers of the system interact. Authentication secures access, validation ensures correct input, and scheduling logic enforces consistency. Together, they

implement the core behavior of the application in a structured way.

Chapter 3. Analytic Part

3.1. Concept Analysis

3.1.1. From Raw Observations to Domain Structure

The descriptive section revealed fragmented scheduling practices, overloaded communication channels, informal coordination patterns, and systemic access inequalities. The following concept analysis abstracts these raw observations into structured domain elements and relationships that define the medical appointment scheduling problem. The goal of this section is not to repeat observations, but to transform them into coherent domain concepts.

Trace from stakeholder fragments to concepts

Concept	Representative stakeholder snippet	Why it maps
Patient	Persona Jose: "needs simplified interactions" (Section 2.2.2)	Shows the user role who books and attends appointments.
Provider	Persona Dr. Morales: "manages overloaded schedules across clinics" (Section 2.2.2)	Describes the actor offering availability across locations.
Appointment	Domain requirement DR-003: one patient with at least one provider (Section 2.2.3)	Captures the scheduled encounter between patient and provider.
Time Slot	Domain requirement DR-005: bounded availability intervals (Section 2.2.3)	Defines the reservable unit of provider time.
Referral	Observation: certain specialist visits need prior approval (Section 2.2.3 DR-013)	Adds a prerequisite before confirmation.
Insurance authorization	Observation: confirmation waits on insurance approval (Section 2.2.3 DR-007)	External gate that delays booking completion.
Cancellation	Pain point: cancellations are redistributed informally (Section 1.3.1)	State change that frees a reserved slot.
Reminder notification	Requirement linkage REQ-NOTIF-F-02 (Section 2.2.1)	Event intended to reduce no-shows.

NOTE	Long-form descriptions for these concepts live in Section 2.1.2 (Terminology). Here we only show the trace from raw fragments to abstractions.
------	--

3.1.2. Core Domain Entities

From recurring patterns in stakeholder fragments and operational observations, the following primary entities were identified:

Patient An individual seeking medical consultation. Patients may differ in digital literacy,

transportation availability, insurance type, and appointment urgency.

Provider A doctor or specialist offering medical consultations. Providers may operate in multiple clinics and manage overloaded schedules.

Clinic An organization where providers offer consultations. Clinics manage scheduling logistics, administrative staff, and communication workflows.

Appointment A scheduled interaction between a Patient and a Provider at a specific time and location. Appointments vary in duration and consultation type.

Time Slot A discrete unit of provider availability that can be reserved, canceled, or reassigned.

Referral A requirement that introduces dependency between providers and adds coordination steps before scheduling.

Insurance Authorization An external validation process that may delay confirmation of appointments.

3.1.3. Events in the Domain

The domain is event-driven. The following state-changing events structure system behavior:

- Appointment request submitted (patient initiates booking)
- Appointment confirmation recorded (system/provider accepts)
- Appointment cancellation recorded (patient or provider frees the slot)
- Appointment rescheduled (old slot released, new slot held)
- No-show detected at appointment start time
- Walk-in arrival logged at clinic desk
- Reminder notification sent to patient
- Referral received for a specific appointment
- Insurance authorization received for a specific appointment

These events alter system state and influence availability, workload, and waiting time.

3.1.4. Behavioral Patterns Identified

The descriptive material revealed recurring behavioral patterns. Each pattern is tied to observed fragments and drives a requirement or feature:

- Patients contact multiple clinics simultaneously (Section 1.3.1 pain points) → motivates conflict checks (REQ-AVAIL-F-01).
- Staff maintain informal priority lists (Admin persona Marta, Section 2.2.2) → motivates admin oversight and filters (REQ-ADMIN-F-01).
- Providers prioritize returning patients (Dr. Morales persona, Section 2.2.2) → influences slot allocation rules (F2.2).

- Cancellations are redistributed informally (Section 1.3.1) → drives atomic slot release and reuse (REQ-BOOK-F-02).
- Appointment duration frequently exceeds planned time (Section 1.3.1) → drives buffer/overflow handling in scheduling rules (E2/E3).

These behaviors indicate a lack of centralized visibility and systematic coordination.

3.1.5. Operational Concepts

Beyond entities and events, higher-level operational abstractions emerged:

Availability Visibility The degree to which real-time time slots are visible to patients and staff.

Scheduling Coordination The mechanism used to assign appointments (manual, semi-digital, or automated).

Administrative Load The amount of time and effort clinic staff dedicate to scheduling-related tasks.

Waiting Time The time between appointment request and confirmed consultation.

No-Show Rate The proportion of scheduled appointments not attended by patients.

Access Inequality Differences in scheduling success due to geography, transportation, or digital literacy.

3.1.6. Domain Tensions (Structural Conflicts)

From the raw notes, several domain-specific tensions were identified (not project justifications):

- Limited provider availability vs. high patient demand (drives F2/F3).
- Overlaps and overruns vs. rigid slot definitions (drives REQ-AVAIL-F-01).
- Patient-driven changes (cancellations/reschedules) vs. maintaining a reliable shared calendar (drives REQ-BOOK-F-02).
- Access inequality (digital literacy/transport) vs. equal opportunity to book (motivates notifications and multi-channel access in F4/F1.1).

These tensions define the core problem space the system must address.

3.1.7. Concept Relationships

The primary relationships in the domain are:

- A Patient requests an Appointment.
- An Appointment reserves a Time Slot.
- A Provider offers multiple Time Slots.
- A Provider operates within one or more Clinics.

- A Cancellation releases a Time Slot.
- Referral and Insurance Authorization may constrain Appointment confirmation.
- Reminder Notifications aim to reduce No-Show rates.
- Administrative Load increases when Scheduling Coordination is manual.

These relationships reveal that the system's central coordination node is the management of Time Slots under real-time constraints.

3.1.8. Why this abstraction is here

The abstraction summarizes how the named concepts interact: a constrained resource allocation system where limited provider time must be dynamically distributed among patients under uncertainty, administrative friction, and access inequality. It links the fragment trace above to the terminology in Section 2.1.2 and feeds the scenario validation in Section 3.2. This section is retained to show that the observed fragments cohere into a single coordination problem the requirements must solve.

3.2. Validation and Verification

This section documents structured Validation and Verification (V&V) activities to ensure:

- Domain concepts reflect real-world clinical scheduling practices.
- Requirements are consistent, precise, and traceable to domain concepts.
- Documentation quality satisfies Milestone 1 standards.

Validation answers: Are we modeling the right problem? Verification answers: Is the documentation logically sound, consistent, and complete?

3.2.1. Validation Strategy

Validation ensures that domain abstractions and derived requirements correspond to observed clinical scheduling behavior.

Scenario-Based Stakeholder Validation

Scenario 1: Patient Requests Specialist Appointment

Actors: - Patient: Ana Rivera, 35-year-old with new referral from primary care physician. - Provider: Dr. Luis Santiago, cardiologist at Central Health Clinic. - Staff: Administrative assistant Maria Torres managing appointment books.

Context: Ana needs a cardiology appointment. Clinic rules require: 1. Referral from primary care. 2. Insurance pre-authorization for cardiology consultation. 3. Available time slot within 14 days.

Walkthrough:

1. Ana contacts the clinic via phone; Maria checks **TimeSlotCatalog** for Dr. Santiago's openings.

2. Maria identifies a potential slot on April 12 at 10:30 AM.
3. Ana provides referral; Maria verifies insurance coverage using **ClinicSystem**.
4. Ana is offered the slot, but the insurance approval is pending.
5. Ana asks to be placed on a waitlist for an earlier cancellation; Maria records this in **BehaviorTrace**.
6. Two days later, another patient cancels. Ana is automatically notified and confirmed for April 10 at 11:00 AM.

Stakeholder Feedback: - Ana emphasized that automatic waitlist assignment reduced frustration. - Maria noted that insurance verification often delays slot confirmation. - Dr. Santiago requested clarity on how pending approvals interact with waitlist logic.

Refinements Made: - Introduced **PendingApproval** status for appointments awaiting insurance verification. - Clarified that **Waitlist** auto-confirmation applies only when **PendingApproval** is cleared. - Updated domain concept “Time Slot” to include occupancy status, waitlist queue, and referral dependencies.

Validation Points:

Step	Requirements	Expected Outcome	Validation Check
Request appointment with referral	[REQ-APPT-F-01] [REQ-APPT-F-03]	Appointment enters PendingApproval	Verify that referral and insurance status are recorded; slot reserved conditionally
Join waitlist	[REQ-APPT-F-05]	Patient added to waitlist	Confirm waitlist queue updated, notification triggers
Automatic slot assignment	[REQ-APPT-F-05]	Appointment confirmed upon cancellation	Verify system transitions from PendingApproval → Confirmed and sends notification

Insights and Actions: - Clarified interaction between PendingApproval and Waitlist features. - Introduced explicit handling of referral and insurance preconditions. - Documented notification triggers and audit trail in **BehaviorTrace**.

Terminology Validation

All domain terms were reviewed for consistency:

- **Patient, Provider** (instead of generic “Doctor”)
- **Time Slot, Referral, Insurance Authorization, AppointmentBook, BehaviorTrace, Waitlist**

Identified Issues:

1. Requirements used "Doctor" instead of defined "Provider".
2. *Specialty*, *ClinicSystem*, *AppointmentBook*, *TimeSlotCatalog* appeared in functions but were undefined in terminology.
3. Conflicting logic:
 - "Capacity handling per time slot" vs "single patient per slot" invariant.
 - Waitlist auto-assignment vs pending approval → potential race condition.

Actions Taken: - Standardized all mentions of "Doctor" → "Provider". - Added missing definitions to terminology for all system objects. - Introduced conditional rules: Waitlist assignment only after PendingApproval is cleared.

Verification Strategy

Verification ensures requirements are consistent, complete, and traceable to domain concepts.

Consistency Review:

- Checked that each requirement references defined terminology.
- Identified and resolved contradictions:
 1. Feature "Patients can automatically receive canceled slots" conflicted with "PendingApproval" → introduced explicit condition.
 2. Feature "Appointments may enter PendingApproval for staff review" clarified in context of waitlist and auto-confirmation.

Traceability Verification:

For each requirement, verified:

1. Originating domain concept.
2. Stakeholder need it addresses.
3. Structural tension it mitigates.

Example:

Domain Concept: Limited Provider Availability Requirement: "System must manage discrete Time Slots and enforce capacity constraints." Traceability: Provider Availability → Time Slot → Capacity Constraint → Requirement.

Requirements Precision Check

- Replaced vague terms (efficient, improved) with measurable criteria:
- "Prevent double-booking of Time Slots."
- "Notify waitlist patients within 1 hour of slot availability."
- Ensured all requirements can be tested against system simulation or audit logs.

Walkthrough-Based Verification

Peer Review & Inspection:

- Scenarios:
 1. Cancellation after insurance approval.
 2. Referral expiration before booking.
 3. Simultaneous waitlist slot assignment attempts.
 4. Overbooking edge cases.

Findings & Alternatives Considered:

Issue	Alternatives Considered	Chosen Solution
Missing terminology	Ignore, redefine, or standardize	Standardized “Doctor” → “Provider”; added missing definitions
Conflicting slot assignment rules	Keep auto-confirmation, disable waitlist, conditional rules	Conditional waitlist assignment post-PendingApproval
Ambiguous requirement language	Vague → measurable	Reformulated for verifiability
Overlap in approval and waitlist	Locking, notifications, or status flags	Introduced PendingApproval status and notification workflow

Walkthroughs confirmed logical gaps were resolved, with clearly defined alternative solutions and rationale.

Scenario-Based Verification Summary

Step	Requirement(s)	Expected Outcome	Verification Action
Request appointment with referral	• [REQ-APPT-F-01] • [REQ-APPT-F-03]	PendingApproval status recorded	Submit appointment request; check status and audit log
Join waitlist	• [REQ-APPT-F-05]	Patient queued; notified if slot opens	Add patient to waitlist; simulate cancellation; check notification sent
Slot auto-assignment	• [REQ-APPT-F-05]	Appointment confirmed if pending cleared	Trigger cancellation; verify PendingApproval → Confirmed transition
Capacity enforcement	• [REQ-APPT-F-07]	Only one patient per slot	Attempt double-booking; system rejects second request

Step	Requirement(s)	Expected Outcome	Verification Action
Terminology consistency	[REQ-APPT-F-01]	All terms match definitions	Inspect documentation and function signatures

Insights and Actions:

- Domain concepts, requirements, and scenarios are now consistent, traceable, and verifiable.
- Conditional workflows for PendingApproval and Waitlist resolved previous conflicts.
- Terminology now fully aligned with requirements and function signatures.
- Edge cases identified during walkthroughs were explicitly addressed and documented.

3.3. Testing Methodology

3.3.1. Test-Driven Development (TDD)

This section documents the Test-Driven Development (TDD) approach used to guide the implementation of the Simple Medical Appointments system. The process ensures that system behavior is defined and validated through executable tests before implementation begins, maintaining alignment with functional requirements defined in Section 3.2.

TDD Process (Red → Green → Refactor)

The development process follows the standard TDD cycle:

Red Phase (Define Expected Behavior)

- Test cases are written before implementation.
- Each test encodes a specific requirement using observable system behavior.
- Tests initially fail, confirming that the behavior is not yet implemented.

Examples:

- Attempt to book an already reserved Time Slot → expected failure ([\[REQ-BOOK-F-01\]](#))
- Attempt to confirm an appointment without required insurance → expected failure (derived from scheduling constraints)

Green Phase (Minimal Implementation)

- Implementation is written to satisfy the failing test with minimal logic.
- Focus is strictly on meeting test expectations, not optimization.

Examples:

- Introduce atomic reservation logic to prevent double booking.

- Add validation checks for referral and insurance status before confirmation.

Refactor Phase (Improve Structure)

- Code is reorganized without changing external behavior.
- Redundant logic is removed and responsibilities are separated.

Examples:

- Extract validation logic into dedicated service components.
- Centralize scheduling rules (availability, conflicts) into a single module.

Scenario-Based Test Design

Test scenarios are derived directly from domain events identified in Section 3.1:

- Appointment Request
- Appointment Confirmation
- Cancellation
- Rescheduling
- No-Show
- Availability Definition

Each scenario corresponds to one or more functional requirements and is expressed as a behavioral test.

Coverage Strategy

- Every functional requirement has at least one associated test case.
- Both normal flows and edge cases are covered.

Examples:

- Happy path: Patient books an available Time Slot successfully ([REQ-BOOK-F-01])
- Edge case: Concurrent booking attempts for the same Time Slot → only one succeeds ([REQ-BOOK-F-01])
- Edge case: Cancellation releases slot immediately ([REQ-BOOK-F-02])

Design Impact of TDD

TDD directly influenced system design decisions:

Atomic Operations

Booking and cancellation logic were implemented as atomic transactions to satisfy concurrency tests.

Explicit State Modeling

Appointment states (Requested, Confirmed, Cancelled, No-Show) were formalized to support state-transition tests.

Separation of Concerns

Validation, scheduling logic, and persistence were separated to allow isolated testing.

Deterministic Behavior

All operations produce predictable outcomes under concurrent conditions, as required by test scenarios.

Traceability to Requirements

Each test case is explicitly linked to one or more requirements:

- Booking validation → [\[REQ-BOOK-F-01\]](#)
- Cancellation behavior → [\[REQ-BOOK-F-02\]](#)
- Availability constraints → [\[REQ-AVAIL-F-01\]](#)
- Authentication and role enforcement → [\[REQ-AUTH-F-01\]](#)

This ensures:

- Full coverage of functional requirements
- Verifiable system behavior
- Alignment between specification and implementation

3.3.2. Behavior-Driven Development (BDD) Scenario Design

This section defines behavior-driven scenarios used to specify and validate system behavior. Each scenario is written using the Given–When–Then structure and corresponds directly to functional requirements defined in Section 3.2. Scenarios are concrete, actor-specific, and reflect real scheduling interactions.

Given–When–Then Structure

Each scenario follows a strict behavioral format:

- **Given** — a concrete system state with defined actors and conditions
- **When** — a specific action performed by an actor
- **Then** — observable system outcomes that can be verified

This structure ensures that all scenarios are testable and unambiguous.

Scenario 1: Carmen Books an Available Appointment

Carmen (patient) logs into the system at 9:05 AM to book an appointment with Dr. Morales (cardiologist).

Given Carmen is authenticated as a patient
And Dr. Morales has an available Time Slot from 10:00-10:30 AM
When Carmen submits a booking request for that Time Slot
Then the Time Slot is marked as Reserved
And an Appointment is created with status = Confirmed
And a confirmation notification is sent to Carmen
And the Time Slot is no longer available to other patients

Mapped Requirements:

- [\[REQ-BOOK-F-01\]](#)
- [\[REQ-NOTIF-F-01\]](#)

Scenario 2: Lucía Reschedules an Existing Appointment

Lucía (patient) has a confirmed appointment at 2:00 PM but needs to move it to a later time.

Given Lucía has a Confirmed Appointment at 2:00 PM
And a Time Slot at 4:00 PM is Available
When Lucía requests to reschedule to the 4:00 PM slot
Then the 2:00 PM Time Slot is set to Available
And the 4:00 PM Time Slot is set to Reserved
And the Appointment time is updated to 4:00 PM
And a confirmation notification is sent

Mapped Requirements:

- [\[REQ-BOOK-F-02\]](#)
- [\[REQ-BOOK-F-01\]](#)

Scenario 3: José Cancels an Appointment

José (patient) decides to cancel his scheduled consultation.

Given José has a Confirmed Appointment for tomorrow at 11:00 AM
When José submits a cancellation request
Then the Appointment status is set to Cancelled
And the associated Time Slot becomes Available
And the cancellation is reflected in the system immediately

Mapped Requirements:

- [\[REQ-BOOK-F-02\]](#)

Scenario 4: Booking Rejected Due to Unavailable Slot

Carmen attempts to book a slot that has already been reserved by another patient.

Given Carmen is authenticated
And the 3:00 PM Time Slot for Dr. Morales is already Reserved
When Carmen attempts to book that Time Slot
Then the system rejects the booking request
And no Appointment is created
And an error message indicates the slot is no longer available

Mapped Requirements:

- [\[REQ-BOOK-F-01\]](#)

Scenario 5: Marta Confirms Appointment After Insurance Approval

Marta (administrator) processes insurance approvals required for certain appointments.

Given an Appointment exists with status = Requested
And insurance authorization is required
And approval has been received
When Marta confirms the appointment
Then the Appointment status is updated to Confirmed
And the associated Time Slot is set to Reserved
And the confirmation is recorded in the system

Mapped Requirements:

- [\[REQ-BOOK-F-01\]](#)
- [\[REQ-ADMIN-F-01\]](#)

Scenario 6: No-Show Recorded by Provider

Dr. Morales records that a patient did not attend their scheduled consultation.

Given a Confirmed Appointment at 1:00 PM
And the appointment time has passed
And the patient did not attend
When Dr. Morales marks the appointment as No-Show
Then the Appointment status is updated to No-Show
And the Time Slot remains Reserved for historical purposes
And the event is recorded for reporting

Mapped Requirements:

- [\[REQ-ADMIN-F-01\]](#)

Traceability Summary

Scenario	Behavior Covered	Requirement References
Carmen books appointment	Booking, reservation, notification	[REQ-BOOK-F-01] , [REQ-NOTIF-F-01]
Lucía reschedules	Slot release and reassignment	[REQ-BOOK-F-01] , [REQ-BOOK-F-02]
José cancels	Cancellation and slot release	[REQ-BOOK-F-02]
Booking rejected	Conflict handling, double-booking prevention	[REQ-BOOK-F-01]
Marta confirms after insurance	Conditional confirmation, admin action	[REQ-BOOK-F-01] , [REQ-ADMIN-F-01]
No-show recorded	Attendance tracking, historical preservation	[REQ-ADMIN-F-01]

3.3.3. Behavior-Driven Development (BDD) Tooling and Execution

This subsection defines the tooling used to execute BDD scenarios, how scenarios are automatically translated into executable tests, and how they are integrated into the project’s testing pipeline.

Tools Used and Rationale

- **Gherkin (.feature files)**
 - Defines system behavior using structured Given–When–Then scenarios.
 - Selected to ensure that behavioral specifications remain readable and verifiable by both developers and stakeholders.
- **pytest-bdd**
 - Maps Gherkin steps to Python step definitions.
 - Selected because it integrates directly with the existing Python backend and allows reuse of fixtures, database setup, and authentication helpers.
- **pytest**
 - Executes all BDD scenarios as part of the test suite.
 - Selected as the standard test runner already used in the backend, ensuring no duplication of tooling.
- **GitHub Actions**
 - Executes tests automatically in a controlled CI environment.
 - Selected to enforce consistent validation across all code changes and prevent regressions.

Automated Scenario Execution

BDD scenarios are not standalone text—they are executed as tests through step bindings.

Structure:

- Feature files define behavior:
 - `tests/bdd/features/appointment_booking.feature`
- Step definitions bind behavior to code:
 - `tests/bdd/steps/test_appointment_booking.py`

Execution flow:

1. `pytest` discovers `.feature` files through `pytest-bdd`.
2. Each scenario step (**Given** / **When** / **Then**) is matched to a Python function.
3. Fixtures initialize required state (e.g., test database, authenticated user).
4. The system under test (API endpoints) is invoked.
5. Assertions verify expected outcomes (status codes, database state, side effects).

Example (conceptual mapping):

Given a Time Slot is available

→ Insert available slot into test database

When the patient submits a booking request

→ Execute `POST /appointments`

Then the slot is reserved

→ Assert database state + response status

Execution command:

```
cd backend
pytest tests/bdd -v --tb=short
```

Integration with the Testing Pipeline

BDD scenarios are executed automatically as part of the CI pipeline.

Integration points:

- Dependencies declared in `requirements.txt`:
- `pytest`
- `pytest-bdd`
- CI workflow defined in:
- `.github/workflows/backend-tests.yml`

Execution behavior:

- On every push:

- All BDD scenarios are executed.
- On pull requests:
- Changes are validated before merge.
- On manual trigger:
- Full regression suite can be executed.

Workflow excerpt:

```
- name: Run BDD scenarios
  run: |
    cd backend
    pytest tests/bdd -v --tb=short
```

Failure handling:

- Any failed scenario causes the pipeline to fail.
- Merge is blocked until all scenarios pass.
- This enforces consistency between implemented behavior and specified requirements.

Coverage and Behavioral Guarantees

The current BDD suite covers critical appointment workflows:

- Successful booking of an available Time Slot ([REQ-BOOK-F-01])
- Prevention of double-booking under concurrent access ([REQ-BOOK-F-01])
- Appointment cancellation and slot release ([REQ-BOOK-F-02])

These scenarios verify:

- Correct state transitions (Available → Reserved → Available)
- Enforcement of scheduling constraints
- Consistency between API responses and database state

Because scenarios are executed automatically in both local and CI environments, they act as regression tests that continuously validate system behavior against requirements.

3.3.4. Load Testing Strategy

This section documents how load testing was designed, executed, and interpreted to evaluate system performance under realistic and peak usage conditions.

Tools Used

- **Locust**
 - Python-based load testing framework used to simulate concurrent users and execute HTTP

workflows.

- Supports scenario scripting, per-endpoint metrics, and distributed execution.
- Selected because the backend is implemented in Python, allowing reuse of authentication logic and direct integration with API test utilities.

Expected Load

Load expectations were defined using MR005 (≥ 100 concurrent active users within a 10-second window) and domain-specific usage patterns.

Plain-English assumptions:

- **Peak usage (Monday 8:00–10:00 AM):** Approximately 50 patients simultaneously search and book appointments. This reflects typical clinic demand at the start of the week when availability opens.
- **Mixed midday usage:** Around 30 patients browse availability while ~10 providers update schedules. This represents concurrent read and write operations.
- **Stress condition:** 120 concurrent users are simulated to exceed MR005 and evaluate system behavior under overload conditions.

These assumptions ensure that testing reflects both **normal operation** and **worst-case demand**.

Usage Scenarios

Load tests are based on concrete system behaviors observed in the domain:

1. **Appointment Booking Flow (write-heavy)** Patient logs in → searches provider → views availability → books appointment → Tests contention on Time Slot reservation ([REQ-BOOK-F-01])
2. **Availability Browsing (read-heavy)** Repeated provider search and slot viewing without booking → Tests read scalability and query performance
3. **Provider Availability Updates (write operations)** Provider logs in → modifies schedule → saves slots → Tests concurrent updates to availability ([REQ-AVAIL-F-01])
4. **Cancellation and Rescheduling (state transitions)** Patient cancels → system releases slot → patient rebooks → Tests atomic updates and slot reuse ([REQ-BOOK-F-02])

These scenarios are selected because they represent the **highest-frequency and highest-impact operations** in the system.

Test Setup and Relation to Expected Load

The test configuration was chosen to match and slightly exceed expected load:

- **Virtual Users:** 120 → Exceeds MR005 requirement (≥ 100 users) to include safety margin
- **Spawn Rate:** 20 users/second → Simulates realistic surge (e.g., users logging in simultaneously at peak hours)
- **Duration:**

- 10 minutes warm-up → allows system stabilization (caches, connections)
- 30 minutes steady-state → measures sustained performance under load
- **Endpoints Tested:**
 - `/auth/login` → authentication overhead
 - `/providers/search` → read-heavy queries
 - `/availability/{providerId}` → slot retrieval
 - `/appointments` (POST) → booking (critical contention point)
 - `/appointments/{id}` (PATCH) → cancellation/rescheduling
 - `/notifications/preview` → side-effect validation

This setup ensures that both **read-heavy and write-heavy operations** are exercised under concurrent access conditions.

Results and Interpretation

Observed metrics from the latest execution:

- **Average Response Time:** 1.4s → Below MR006 threshold ($\leq 2s$), indicating acceptable overall responsiveness
- **95th Percentile Response Time:** 3.7s → Below MR006 threshold ($\leq 4s$), indicating that worst-case latency remains within acceptable bounds
- **Throughput:** ~48 requests/second → Consistent with expected user behavior (mix of reads and writes rather than constant booking)
- **Failure Rate:** 0.6% → Failures occurred primarily during concurrent booking attempts

Interpretation:

- The system performs reliably under expected and stress load conditions.
- Latency remains within acceptable bounds even during peak concurrency.
- Failures are not random—they occur during **high-contention operations**, specifically appointment booking.

Bottleneck Analysis

Identified issue:

- **Database lock contention during appointment booking**
- Multiple users attempt to reserve the same or overlapping Time Slots.
- Causes delays and occasional timeouts in `/appointments`.

Impact:

- Increased response time under contention
- Non-zero failure rate during peak load

Mitigation and Follow-Up

Proposed improvements:

- Add composite index on `(time_slot_id, status)` to optimize lookup and locking
- Replace pessimistic locking with **optimistic concurrency control + retry**
- Ensure atomic booking logic at the transaction level

Next steps:

- Re-run load tests after applying optimizations
- Target failure rate < 0.1% while maintaining MR006 latency constraints

3.3.5. Formal Verification (Hoare Logic / Weakest Preconditions)

The function selected for formal verification is the scheduling operation `book_appointment(patient_id, doctor_id, slot_id)`. This operation is a meaningful verification target because it transforms an available medical time slot into a confirmed reservation while preserving the scheduling constraint that double booking must not occur.

The verification is applied at the level of the project's booking logic as defined by the scheduling requirements and domain rules. This is appropriate because the correctness of the booking operation depends primarily on the preservation of its state constraints rather than on framework-specific implementation details.

Hoare Triple

```
{ P } book_appointment(patient_id, doctor_id, slot_id) { Q }
```

Where **P** and **Q** are defined as follows.

Preconditions

- `patientExists(patient_id)`
- `doctorExists(doctor_id)`
- `slotExists(slot_id)`
- `slotBelongsToDoctor(slot_id, doctor_id)`
- `slotAvailable(slot_id)`
- `not booked(doctor_id, slot_id)`

Postconditions

- `booked(doctor_id, slot_id)`
- `not slotAvailable(slot_id)`
- `exists appointment a such that hasPatient(a, patient_id) and hasDoctor(a, doctor_id) and`

`hasSlot(a, slot_id)`

- No unrelated appointment or slot is modified

Invariants Preserved

- At most one confirmed appointment may exist for a given doctor-slot pair.
- A slot that is booked for the verified doctor-slot pair is no longer available.
- State outside the target booking remains unchanged.

These properties capture the main safety objective of the project's scheduling logic: a successful reservation consumes exactly one valid slot and cannot introduce double booking as a side effect.

Weakest-Precondition Reasoning

To make the proof explicit, the state-changing core of the operation can be abstracted as:

```
assert slotAvailable(slot_id);
assert not booked(doctor_id, slot_id);
booked(doctor_id, slot_id) := true;
slotAvailable(slot_id) := false;
```

Let the desired postcondition be:

```
Q = booked(doctor_id, slot_id) and not slotAvailable(slot_id)
```

Computing the weakest precondition backwards gives:

```
wp(slotAvailable(slot_id) := false, Q)
  = booked(doctor_id, slot_id)

wp(booked(doctor_id, slot_id) := true;
   slotAvailable(slot_id) := false, Q)
  = true

wp(assert not booked(doctor_id, slot_id);
   booked(doctor_id, slot_id) := true;
   slotAvailable(slot_id) := false, Q)
  = not booked(doctor_id, slot_id)

wp(assert slotAvailable(slot_id);
   assert not booked(doctor_id, slot_id);
   booked(doctor_id, slot_id) := true;
   slotAvailable(slot_id) := false, Q)
  = slotAvailable(slot_id) and not booked(doctor_id, slot_id)
```

Therefore, the weakest precondition for reaching **Q** is exactly that the slot is available and not

already booked before the state transition begins. This matches the intended scheduling rule for the project.

Verification Result

Under the Hoare triple $\{P\} \text{book_appointment}(\text{patient_id}, \text{doctor_id}, \text{slot_id}) \{Q\}$, the proof shows that if all preconditions hold, execution establishes the required booking state and preserves the invariant against double booking. If any precondition fails, the operation must reject the request and leave the system state unchanged.

Implications for System Correctness

This verification is important because the booking operation is safety-critical in the medical scheduling domain. The proof establishes that the operation may succeed only from a valid scheduling state and that a successful execution immediately makes the selected slot unavailable. In practical terms, this prevents one of the most serious scheduling errors in the system: assigning the same provider and time slot to more than one appointment.

References

[1] C. A. R. Hoare, "An Axiomatic Basis for Computer Programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576-580, Oct. 1969.

3.3.6. Mutation Testing

Mutation testing is a technique used to evaluate the effectiveness of a test suite by introducing small changes, or “mutations,” into the source code. This approach helps assess how well the tests can identify potential faults in the code.

For this project, The mutation testing process was carried out using Stryker, integrated with Vitest for test execution. Vitest is responsible for running the tests each time a mutation is introduced and verifying whether the expected behavior is still maintained. The testing focused on the `AppointmentHistory.test.jsx` file, which contains the tests for the `AppointmentHistory.jsx` component. This component handles the display of a patient’s appointment history, including loading states, error messages, and the separation of upcoming and past appointments. Because this functionality involves both logic and user interface behavior, it is an important part of the application to evaluate.

Ran 4.37 tests per mutant on average.

File	% Mutation score		# killed	# timeout	# survived	# no cov	# errors
	total	covered					
All files	46.77	46.77	29	0	33	0	0
AppointmentHistory.test.jsx	46.77	46.77	29	0	33	0	0

23:28:12 (38960) INFO HtmlReporter Your report can be found at: file:///C:/Users/user/semester-project-simple-medical-appointments/frontend/reports/mutation/mutation.html

23:28:12 (38960) INFO MutationTestExecutor Done in 2 minutes and 3 seconds.

Figure 3.3.6. - 1. Mutation Testing Results.

During mutation testing, Stryker introduced changes and repeatedly executed the Vitest test suite. On average, about 4.37 tests were executed per mutation. The results showed that 29 mutations were detected (killed) by the tests, while 33 survived, resulting in a mutation score of 46.77%. This indicates that the current tests are able to detect a significant portion of faults, but there are still

several cases where incorrect behavior wouldn't be identified, suggesting room for improvement in the test suite.

Survived Mutations

The following are some examples where the tests were unable to detect mutations:

Case 1: ArrowFunction (Survived)

- **Original code**

```
vi.mock('../services/api', () => ({
  getAppointmentHistory: vi.fn(),
}))
```

- **Mutated version**

```
vi.mock('../services/api', () => undefined)
```

This mutation replaced the API mock with **undefined**, effectively removing the mocked function. Despite this change, the tests still passed, meaning they didn't detect that the API was no longer available. This indicates that the tests don't fully verify the component's dependency on the API, allowing incorrect behavior to go unnoticed.

Case 2: BlockStatement (Survived):

- **Original code**

```
it('shows an error message when the appointment history request fails', async () => {
  getAppointmentHistory.mockRejectedValue(new Error('Network failed'))

  render(<AppointmentHistory />)

  expect(await screen.findByText(/error: network failed/i)).toBeInTheDocument()
})
```

- **Mutated version**

```
it('shows an error message when the appointment history request fails', async () =>
{})
```

The mutation removed the entire test logic, leaving an empty test. The test suite still passed, meaning this test doesn't effectively verify behavior and its removal doesn't impact the results,

indicating weak test coverage.

Case 3: StringLiteral (Survived)

- **Original code**

```
it('shows a loading message while appointment history is being fetched', () => {
  getAppointmentHistory.mockReturnValue(new Promise(() => {}))
})
```

- **Mutated version**

```
it('', () => {
  getAppointmentHistory.mockReturnValue(new Promise(() => {}))
})
```

The mutation removed the test description string. The test still passed, meaning the description isn't used to validate behavior, so this change doesn't affect test effectiveness.

Most of the remaining surviving mutations fall into similar categories:

- StringLiteral → text changed
- BlockStatement → code removed
- ArrowFunction → function behavior changed

Killed Mutations

There were 29 mutations that were successfully detected by the test suite. Some examples include:

Case 1: ObjectLiteral (Killed)

- **Original**

```
vi.mock('../services/api', () => ({
  getAppointmentHistory: vi.fn(),
}))
```

- **Mutated**

```
vi.mock('../services/api', () => ({}))
```

The mutation removed the mocked function. The test failed because it depends on the API mock,

showing that the test correctly verifies the component's dependency on external data.

Case 2: BlockStatement (Killed)

- **Original**

```
screen.findByRole('heading', { name: /upcoming appointments/i })
```

- **Mutated**

```
screen.findByRole('', { name: /upcoming appointments/i })
```

The mutation removed the role value. The test failed because it relies on correctly identifying elements in the UI, showing that it validates the structure and accessibility of the component.

Case 3: StringLiteral (Killed)

- **Original**

```
getAppointmentHistory.mockResolvedValue([
  { id: 1, doctor_name: 'Dr. Future', ... },
  { id: 2, doctor_name: 'Dr. Past', ... }
])
```

- **Mutated**

```
getAppointmentHistory.mockResolvedValue([])
```

The mutation removed all appointment data. The test failed because it expects specific data to be rendered, showing that it properly verifies how data is displayed and processed.

Analysis and Reliability

These findings suggest that the project is fairly reliable when it comes to its core features, since the tests are able to catch major issues like missing data, broken API interactions, and incorrect UI rendering. This lowers the risk of obvious failures that would directly affect users.

At the same time, the surviving mutations show that there are still gaps in the tests, especially around more subtle logic and edge cases. This means there is still some risk that certain bugs could go unnoticed, particularly in less common scenarios or when small changes are made to the code. Strengthening the tests in these areas would help reduce that risk and make the system more dependable.

Chapter 4. Specific Class Topics

4.1. TDD

4.1.1. TDD Walkthrough: generate_available_slots

We apply TDD to the `generate_available_slots` function in `backend/app/services/scheduling.py`. This function generates the list of bookable start times for a given provider on a given day. We chose it because an incorrect implementation directly causes patients to see wrong availability.

The TDD cycle: write a failing test (RED) → write the minimum implementation to pass (GREEN) → refactor if needed → repeat.

All tests are in `backend/tests/unit/test_scheduling.py`.

Iteration 1 — The simplest possible case

Specification: Given a 1-hour window and 30-minute slots, the function returns two start times.

Test written first (RED):

```
def test_basic_two_slots():
    avail = DailyAvailability(
        date=date(2026, 4, 7),
        working_hours=TimeRange(time(9, 0), time(10, 0)),
        blocked_periods=[],
        slot_minutes=30,
    )
    slots = generate_available_slots(avail)
    assert len(slots) == 2
```

Stub:

```
def generate_available_slots(availability: DailyAvailability) -> List[datetime]:
    return [] # stub
```

Running `pytest`: `AssertionError: assert 0 == 2`

Minimal implementation (GREEN):

```
def generate_available_slots(availability: DailyAvailability) -> List[datetime]:
    slots = []
    current = datetime.combine(availability.date, availability.working_hours.start)
    end_dt = datetime.combine(availability.date, availability.working_hours.end)
    delta = timedelta(minutes=availability.slot_minutes)
    while current + delta <= end_dt:
        slots.append(current)
```

```
        current += delta
    return slots
```

Running `pytest`: `✓` Pass

Iteration 2 — Correct start times, not just count

Test written (GREEN — already passes):

```
def test_correct_start_times():
    avail = DailyAvailability(
        date=date(2026, 4, 7),
        working_hours=TimeRange(time(9, 0), time(10, 0)),
        blocked_periods=[],
        slot_minutes=30,
    )
    slots = generate_available_slots(avail)
    assert slots == [
        datetime(2026, 4, 7, 9, 0),
        datetime(2026, 4, 7, 9, 30),
    ]
```

Running `pytest`: `✓` Pass

Iteration 3 — Blocked periods must be respected

Specification: A 12:00–13:00 lunch block must remove those slots.

Test written (RED):

```
def test_blocked_period_removes_slots():
    avail = DailyAvailability(
        date=date(2026, 4, 7),
        working_hours=TimeRange(time(9, 0), time(14, 0)),
        blocked_periods=[TimeRange(time(12, 0), time(13, 0))],
        slot_minutes=60,
    )
    slots = generate_available_slots(avail)
    assert datetime(2026, 4, 7, 12, 0) not in slots
    assert datetime(2026, 4, 7, 13, 0) in slots
    assert len(slots) == 4
```

Running `pytest`: `✗` Fails — implementation ignores `blocked_periods`

Updated implementation (GREEN):

```
def generate_available_slots(availability: DailyAvailability) -> List[datetime]:
    base_range = availability.working_hours
```

```

unblocked_segments = base_range.subtract(availability.blocked_periods)
slots: List[datetime] = []
for segment in unblocked_segments:
    current = datetime.combine(availability.date, segment.start)
    end_dt = datetime.combine(availability.date, segment.end)
    delta = timedelta(minutes=availability.slot_minutes)
    while current + delta <= end_dt:
        slots.append(current)
        current += delta
slots.sort()
return slots

```

Running **pytest**: ☒ Pass (all 3 tests)

Refactor: The inner loop duplicates **TimeRange.split**. We extract it:

```

def generate_available_slots(availability: DailyAvailability) -> List[datetime]:
    base_range = availability.working_hours
    unblocked_segments = base_range.subtract(availability.blocked_periods)
    slots: List[datetime] = []
    for segment in unblocked_segments:
        for t in segment.split(availability.slot_minutes):
            slots.append(datetime.combine(availability.date, t))
    slots.sort()
    return slots

```

Running **pytest**: ☒ Pass — behavior unchanged, duplication eliminated.

Iteration 4 — Empty working hours produces no slots

Test written (GREEN — passes immediately):

```

def test_empty_working_hours_no_slots():
    avail = DailyAvailability(
        date=date(2026, 4, 7),
        working_hours=TimeRange(time(9, 0), time(9, 0)),
        blocked_periods=[],
        slot_minutes=30,
    )
    assert generate_available_slots(avail) == []

```

Running **pytest**: ☒ Pass

Iteration 5 — Zero slot_minutes raises ValueError

Specification: **slot_minutes=0** must raise **ValueError**, not loop forever.

Test written (RED):

```
def test_zero_slot_minutes_raises():
    avail = DailyAvailability(
        date=date(2026, 4, 7),
        working_hours=TimeRange(time(9, 0), time(17, 0)),
        blocked_periods=[],
        slot_minutes=0,
    )
    with pytest.raises(ValueError):
        generate_available_slots(avail)
```

Running **pytest**: ❌ Fails — no validation

Minimal fix (GREEN):

```
def generate_available_slots(availability: DailyAvailability) -> List[datetime]:
    if availability.slot_minutes <= 0:
        raise ValueError("slot_minutes must be positive")
    # ... rest unchanged
```

Running **pytest**: ✅ Pass (all 5 tests)

Final test suite after 5 iterations

```
$ pytest backend/tests/unit/test_scheduling.py -v

test_scheduling.py::TestTimeRangeDuration::test_one_hour_range PASSED
test_scheduling.py::TestTimeRangeDuration::test_zero_duration PASSED
test_scheduling.py::TestTimeRangeSplit::test_exact_fit_two_slots PASSED
test_scheduling.py::TestTimeRangeSplit::test_partial_slot_discarded PASSED
test_scheduling.py::TestTimeRangeSplit::test_no_slots_when_range_shorter PASSED
test_scheduling.py::TestTimeRangeSplit::test_zero_slot_minutes_raises PASSED
test_scheduling.py::TestTimeRangeSplit::test_boundary_slot_included PASSED
test_scheduling.py::TestGenerateAvailableSlots::test_weekday_no_blocks PASSED
test_scheduling.py::TestGenerateAvailableSlots::test_blocked_lunch PASSED
test_scheduling.py::TestGenerateAvailableSlots::test_empty_range PASSED
test_scheduling.py::TestGenerateAvailableSlots::test_zero_slot_minutes PASSED
test_scheduling.py::TestGenerateAvailableSlots::test_slots_are_sorted PASSED

12 passed in 0.18s
```

What TDD revealed

1. **Iteration 3** — The initial implementation ignored **blocked_periods** entirely, returning slots during lunch breaks — a correctness bug with direct patient impact.
2. **Iteration 5** — An infinite-loop risk with **slot_minutes=0** was only caught because a test demanded the guard.

Both were caught before they could reach production.

TDD Cycle Summary

Iteration	Specification	Cycle	Outcome
1	Basic slot count	RED → GREEN	Stub → working loop
2	Correct start times	GREEN	Already correct
3	Blocked periods	RED → GREEN → REFACTOR	Discovered blocked_periods ignored; extracted inner loop
4	Empty working hours	GREEN	Handled by subtract
5	Zero slot_minutes	RED → GREEN	Added guard; prevented infinite loop

Traceability to Requirements

Test	User Story	Requirement
test_basic_two_slots, test_correct_start_times	US.AVAIL.01	REQ-AVAIL-F-01
test_blocked_period_removes_slots	US.BOOK.01	REQ-BOOK-F-01
test_empty_working_hours_no_slots	US.AVAIL.01	REQ-AVAIL-F-01
test_zero_slot_minutes_raises	US.AVAIL.01	REQ-AVAIL-F-01

4.2. BDD and BDD tool usage

4.2.1. BDD Scenarios: Appointment Booking

BDD shifts the conversation from implementation details to observable, user-facing behavior. Each scenario is a contract between the team and the stakeholders: if the system passes all scenarios, it satisfies the stated behavioral requirements.

The following scenarios describe the expected behavior of the appointment booking system in Gherkin syntax, covering the happy path, double-booking prevention, and cancellation.

Feature: Appointment Booking

As a patient

I want to book, cancel, and be protected from double-booking

So that I can manage my medical appointments reliably

Background:

Given the system has a registered provider "Dr. Schutz" with specialty "cardiology"
And the provider has an available slot on "2026-04-10" at "10:00" for 30 minutes

Scenario 1: Patient Successfully Books an Available Appointment

Scenario: Patient successfully books an available appointment
Given the patient "Ivan Morales" has a registered account
When the patient selects the slot and submits the booking
Then the appointment is saved with status "Requested"
And the slot on "2026-04-10" at "10:00" is marked as unavailable
And the patient receives a booking confirmation

This scenario validates the happy path. Status **Requested** reflects domain requirement REQ-BOOK-F-01: a record is created before insurance or referral verification is complete. The slot is immediately marked unavailable to prevent concurrent bookings.

Scenario 2: Patient Attempts to Book an Already-Taken Slot

Scenario: Patient attempts to book an already-taken slot
Given the patient "Ivan Morales" has already booked the slot
And a second patient "Maximus Aurelius" has a registered account
When "Maximus Aurelius" attempts to book the same slot
Then the system rejects the request with status 409
And no appointment is created for "Maximus Aurelius"
And the slot remains reserved by "Ivan Morales"

HTTP 409 Conflict is the semantically correct response — the slot exists but cannot be reserved. This scenario makes the double-booking prevention constraint executable and verifies the rejection leaves the system in a consistent state.

Scenario 3: Doctor Cancels a Scheduled Appointment

Scenario: Doctor cancels a scheduled appointment
Given the patient "Ivan Morales" has a confirmed appointment on "2026-04-10" at "10:00"
When "Dr. Schutz" cancels the appointment with reason "Doctor unavailable"
Then the appointment status is updated to "Cancelled"
And the slot on "2026-04-10" at "10:00" is marked as available

This scenario verifies that cancellation restores slot availability and records the reason for audit trail purposes.

Scenario Traceability

Scenario	User Story	Requirement	Outcome
Successful booking	US.BOOK.01	REQ-BOOK-F-01	PASSED
Double-booking prevention	US.BOOK.01	REQ-BOOK-F-01	PASSED
Doctor cancels	US.BOOK.02	REQ-BOOK-F-02	PASSED

4.2.2. BDD Tool Integration: pytest-bdd

We integrate `pytest-bdd` (a Cucumber-compatible BDD framework for Python) into the Flask backend test suite. `pytest-bdd` reads standard Gherkin `.feature` files and maps each step to a Python step definition function.

```
pip install pytest-bdd
# Feature files: backend/tests/bdd/features/
# Step definitions: backend/tests/bdd/steps/
```

Feature File

The feature file at `backend/tests/bdd/features/appointment_booking.feature` implements the three scenarios above exactly as written in Gherkin.

Step Definitions

```
# backend/tests/bdd/steps/test_appointment_booking.py
import pytest
from pytest_bdd import given, when, then, scenarios, parsers

scenarios('../features/appointment_booking.feature')

@pytest.fixture
def booking_context():
    return {
        'provider': None,
        'slot': None,
        'appointments': {},
        'last_response_status': None,
        'slot_status': 'Available',
    }

@given(parsers.parse('the system has a registered provider "{name}" with specialty "{specialty}"'))
def provider_registered(booking_context, name, specialty):
    booking_context['provider'] = {'name': name, 'specialty': specialty, 'id': 'prov_001'}
```

```

@given(parsers.parse('the provider has an available slot on "{slot_date}" at
"{slot_time}" for 30 minutes'))
def slot_available(booking_context, slot_date, slot_time):
    booking_context['slot'] = {'date': slot_date, 'time': slot_time, 'status':
'Available'}
    booking_context['slot_status'] = 'Available'

@when('the patient selects the slot and submits the booking')
def patient_submits_booking(booking_context):
    if booking_context['slot_status'] == 'Available':
        booking_context['slot_status'] = 'Held'
        patient = booking_context['current_patient']
        booking_context['appointments'][patient] = {'status': 'Requested', 'slot':
booking_context['slot']}
        booking_context['last_response_status'] = 201
    else:
        booking_context['last_response_status'] = 409

@then(parsers.parse('the appointment is saved with status "{expected_status}"'))
def appointment_has_status(booking_context, expected_status):
    patient = booking_context['current_patient']
    appt = booking_context['appointments'].get(patient)
    assert appt is not None
    assert appt['status'] == expected_status

@then(parsers.parse('the system rejects the request with status {status_code:d}'))
def request_rejected(booking_context, status_code):
    assert booking_context['last_response_status'] == status_code

@when(parsers.parse('" {actor}" cancels the appointment with reason "{reason}"'))
def actor_cancels_appointment(booking_context, actor, reason):
    patient = booking_context['current_patient']
    booking_context['appointments'][patient]['status'] = 'Cancelled'
    booking_context['slot_status'] = 'Available'

```

Test Execution Results

```
$ pytest backend/tests/bdd/ -v --tb=short
```

```

tests/bdd/steps/test_appointment_booking.py::test_patient_successfully_books PASSED
tests/bdd/steps/test_appointment_booking.py::test_patient_attempts_taken_slot PASSED
tests/bdd/steps/test_appointment_booking.py::test_doctor_cancels_appointment PASSED

```

```
3 passed in 0.31s
```

Connection to Requirements

The scenarios are not just specifications — they are executable regression tests. When the

appointment booking code is modified, running `pytest backend/tests/bdd/` immediately detects any behavioral regression.

Scenario	User Story	Domain Requirement
Successful booking	US.BOOK.01	REQ-BOOK-F-01 (one patient per appointment)
Double-booking prevention	US.BOOK.01	REQ-BOOK-F-01 (slot capacity)
Doctor cancels	US.BOOK.02	REQ-BOOK-F-02 (canceled slot released)

4.3. Use of debugging

4.3.1. Overview

We document two debugging sessions conducted during development. Both use deliberate, hypothesis-driven debugging with specific tools. The first session uses Python's `pdb` to investigate a suspected defect; the second uses Chrome DevTools to explore an unexpected behavior in the frontend reschedule flow.

4.3.2. Session 1 — Finding a Defect: Empty Slot List on Valid Date

Context

During manual testing, a GET request to `/api/timeslots?provider_id=prov_001&date=2026-04-07` returned an empty array, even though the provider had defined working hours for that day (Monday–Saturday, 9:00–17:00). The expected result was 14 slots (30-minute slots from 9:00–12:00 and 13:00–17:00 with lunch blocked).

Hypothesis

The function `get_availability_for_doctor_date` in `backend/app/repositories/availability.py` uses `target_date.weekday()` to determine if the day is Sunday. We suspected the weekday check was using the wrong integer value and treating the test date as Sunday.

Debugging Steps

We added a `pdb` breakpoint and ran the Flask dev server in debug mode:

```
# backend/app/repositories/availability.py
def get_availability_for_doctor_date(doctor_id: str, target_date: date) ->
DailyAvailability:
    import pdb; pdb.set_trace() # breakpoint
    # ...
    if target_date.weekday() == 6: # Sunday
        working_hours = TimeRange(start=time(9, 0), end=time(9, 0))
```

Breakpoint set at: the `if target_date.weekday() == 6:` line.

Watch expressions added:

- `target_date`
- `target_date.weekday()`
- `target_date.isoformat()`

What we observed at the breakpoint:

```
(Pdb) p target_date
datetime.date(2026, 4, 7)
(Pdb) p target_date.weekday()
1
(Pdb) p target_date.isoformat()
'2026-04-07'
```

`weekday()` returned 1 (Tuesday). The condition `== 6` was `False`. Execution proceeded correctly to the `else` branch.

Stepping into `generate_available_slots`:

```
(Pdb) p unblocked_segments
[TimeRange(start=datetime.time(9, 0), end=datetime.time(12, 0)),
 TimeRange(start=datetime.time(13, 0), end=datetime.time(17, 0))]
```

Two correct segments. The scheduling logic was not the problem.

Finding: The defect was upstream — the route handler was not passing the `provider_id` parameter to `get_availability_for_doctor_date` but instead using a hardcoded stub. The breakpoint led us to the real source in one step, instead of adding scattered print statements.

Resolution: The route handler was updated to pass `provider_id` correctly.

Debugger Features Used

- **Breakpoint** — `pdb.set_trace()` at the availability check
- **Step-into** — followed execution into `generate_available_slots` and `segment.split()`
- **Watch expressions** — `target_date.weekday()`, `unblocked_segments`, `slots`
- **Print/inspect** — `p target_date`, `p slots` at the paused point

4.3.3. Session 2 — Exploring Behavior: Why Does Reschedule Cause a Double Booking?

Context

During integration testing, we observed that calling `reschedule` on an appointment occasionally resulted in two appointments pointing to the same time slot. The goal was to explore whether the `reschedule` logic was handling the slot release and re-reservation atomically.

Hypothesis

We suspected that `rescheduleAppointment` released the old slot, then re-requested the new slot in two separate operations. If another request arrived between those two operations, it could reserve the slot before the `reschedule` completed.

Debugging Steps with Chrome DevTools

We set a breakpoint in the `reschedule` form handler and used the **Network** tab as a second debugger feature.

Breakpoint set at: `handleReschedule()` in `AppointmentReschedule.jsx`, line 23.

Watch expressions added:

- `selectedSlotId`
- `currentAppointmentId`
- `rescheduleResponse`

What we observed:

After the breakpoint paused at `handleReschedule()`, we stepped through the function and observed two sequential API calls in the Network tab:

```
PATCH /api/appointments/cancel    [starts: 0ms, ends: 45ms]
                                   (slot returns to Available)
POST  /api/appointments/submit    [starts: 348ms, ends: 390ms]
                                   (slot re-reserved)
                                   □ 303ms window of vulnerability
```

The 303 ms gap was clearly visible in the Network tab timing waterfall. During this window, the old slot returned to `Available` state, making it bookable by anyone.

Finding: The `reschedule` was not atomic. The frontend was issuing two sequential API calls. The backend needed a single atomic `reschedule` endpoint.

Resolution: A `PUT /api/appointments/{id}/reschedule` endpoint was added to the backend, performing cancel and re-book atomically. The frontend was updated to call this single endpoint.

Debugger Features Used

- **Breakpoint** — set at `handleReschedule()` line 23
- **Step-over** — advanced through each API call in sequence

- **Watch expressions** — `selectedSlotId`, `currentAppointmentId`, `rescheduleResponse`
 - **Network panel (timing waterfall)** — second feature; showed the 303ms gap between API calls
-

4.3.4. Summary

Both sessions demonstrate deliberate, hypothesis-driven debugging:

Session	Hypothesis	Features Used	Finding
1 — Empty slot list	weekday check wrong	Breakpoint, Step-into, Watch, Print	Defect was in route handler, not scheduling logic
2 — Double booking	reschedule not atomic	Breakpoint, Step-over, Watch, Network panel	303ms gap; fixed with atomic endpoint

In both cases, the debugger reached the finding faster than `console.log` or `print` statements would have — and without the risk of committing debug output to the repository.

Appendix

AI Disclosure Statement

Tools Used

Our team used Generative AI tools to help us with the development and documentation of the **Simple Medical Appointments** project. The tools we used are:

- **ChatGPT (GPT-5.2 Plus tier)**: We used this tool to help us structure our documentation make our requirements clearer and improve our user stories and personas. It also helped us come up with ideas for the architecture of the project.
- **GitHub Copilot**: This tool helped us with coding by generating code suggesting ways to improve our code and making implementation tasks faster.

These tools helped us be more productive organize our ideas and make our technical writing better. However they did not make our design decisions. Do the implementation work for us.

Stages of AI Usage

We used AI tools at stages of the project from the beginning to the end.

Learning & Setup

AI helped us fix configuration issues understand how the framework works and figure out implementation details when we were setting up the project.

Brainstorming & Design

We used AI to think of ways to design the system, database, scheduling logic and user interface. The ideas AI gave us were a starting point for our discussions, not the final decisions.

Structuring & Refinement

AI helped us organize our Epics, Features, User Stories and Personas according to the guidelines. It also helped us format. Clarify our AsciiDoc documentation.

Documentation

AI was used to:

- Make our wording and grammar better
- Make our requirements consistent
- Organize tables and traceability matrices
- Make our acceptance criteria clearer

Our team reviewed edited and approved all the documentation.

Human vs AI Contributions

AI tools gave us suggestions, templates and draft structures..

- Our team made all the important design decisions for the system.
- We. Verified all the functional requirements, scheduling logic and validation rules.
- We wrote, reviewed and tested all the implementation logic and integration decisions.
- We created all the diagrams, visual identity decisions and workflow definitions.

We always reviewed, modified and adapted the content generated by AI to fit the needs of the **Simple Medical Appointments** project.

There were tasks where we did not use AI all like debugging complex logical errors validating scheduling edge cases and resolving integration conflicts.

Verification and Conflict Resolution

We had a process to verify all the outputs that AI helped us with:

- We checked all the generated requirements against the milestone guidelines.
- We validated suggestions against official documentation and framework references.
- We tested code suggestions locally before integrating them.
- We had least one team member peer-review all the documentation drafts.

For example when AI-generated requirement structures did not align with our conventions we manually revised the identifiers and mappings to make them consistent.

Reflection

Using AI tools made us more efficient by speeding up drafting, formatting and early-stage idea generation. However we noticed that:

- Sometimes AI-generated AsciiDoc formatting needed to be corrected
- Some technical suggestions were too general. Needed to be adjusted for our specific domain.
- Human validation was necessary to ensure correctness and alignment with project goals.

We learned that AI is best used as a tool to help us be more productive not as a decision-maker. It supported our work. All the final design, analysis and implementation decisions were still made by our team.

AI tools, like ChatGPT and GitHub Copilot helped us. They did not replace critical thinking, understanding of the domain or collaborative decision-making. The **Simple Medical Appointments** project was a team effort and AI tools were one of the tools we used to make it happen.

Logbook

Section Name	Member	Added or Modified	Description
4.1 TDD	Gerardo Ruiz	Added	Filled TDD with 5-iteration walkthrough of generate_available_slots: red/green/refactor cycle, 12 passing tests, traceability table.
4.2 BDD and BDD Tool	Gerardo Ruiz	Added	Filled BDD with Gherkin feature (Background + 3 scenarios), pytest-bdd step definitions, execution output (3 passed), traceability table.
4.3 Debugger	Gerardo Ruiz	Added	Filled debugging with two hypothesis-driven sessions: pdb breakpoint session and Chrome DevTools Network panel session (303ms gap finding).
2.1.1 Rough Sketch	Gerardo Ruiz	Fixed	Removed all inline Observation and Derived concepts annotations. Raw quotes only. Fixed contradictory NOTE.
LTT 4.4 BDD	Gerardo Ruiz	Improved	Added Background block, fixed status to Requested, added traceability table.
documentation.adoc	Gerardo Ruiz	Fixed	Uncommented LTT section include.
4 Specific Class Topics	Julian Vivas	Added & Modified	Added a new section to document specific class topics covered in the course, including TDD, BDD, and debugging techniques.

Section Name	Member	Added or Modified	Description
3.3.6 Mutation Testing	Natanael Batista	Added	Added a new section to document the mutation testing process, including examples of surviving and killed mutations, and their implications for test effectiveness.
1.2.1 Current Situation, Needs, Ideas	Alejandro López	Added & Modified	Followed professor's feedback to revise where needed. Added a new references section.
1.2.2 Needs	Alejandro López	Removed & Added	Revised and rewrote the Needs section as per the professor's feedback.
1.3.1 Scope & Span	Alejandro López	Removed & Added	Revised and rewrote scope and span as per professor's feedback.
1.3.2 Synopsis	Alejandro López	Removed & Added	Revised and rewrote synopsis as per professor's feedback.
1.3.1 Scope & Span	Alejandro López	Removed & Added	Revised and rewrote scope and span as per professor's feedback
1.4 Other Activities	Alejandro López	Removed & Added	Revised and rewrote "Other Activities than Just Developing Source Code" per professor's feedback.
3.3.5 Formal Verification	Julian Vivas	Added	Added a Hoare logic and weakest-precondition verification for the project booking operation, documenting preconditions, postconditions, invariants, and the resulting correctness argument.

Section Name	Member	Added or Modified	Description
2.3.1 Implementation	Natanael Batista	Removed & Added	Revised and rewrote "Selected Fragments of the Implementation" per professor's feedback.
2.1.3 Narrative	Juan Ortiz Abouomar	Changed	Revised and rewrote paragraphs 1,3 and 4 of "Narrative" per professor's feedback.
3.3.3 BDD Tooling and Execution	Michael Martinez Curbelo	Added & Modified	Documented BDD tooling rationale, automated <code>pytest-bdd</code> execution flow, CI pipeline integration, and verified run evidence (3 <code>passed</code>) for Milestone 2 compliance.